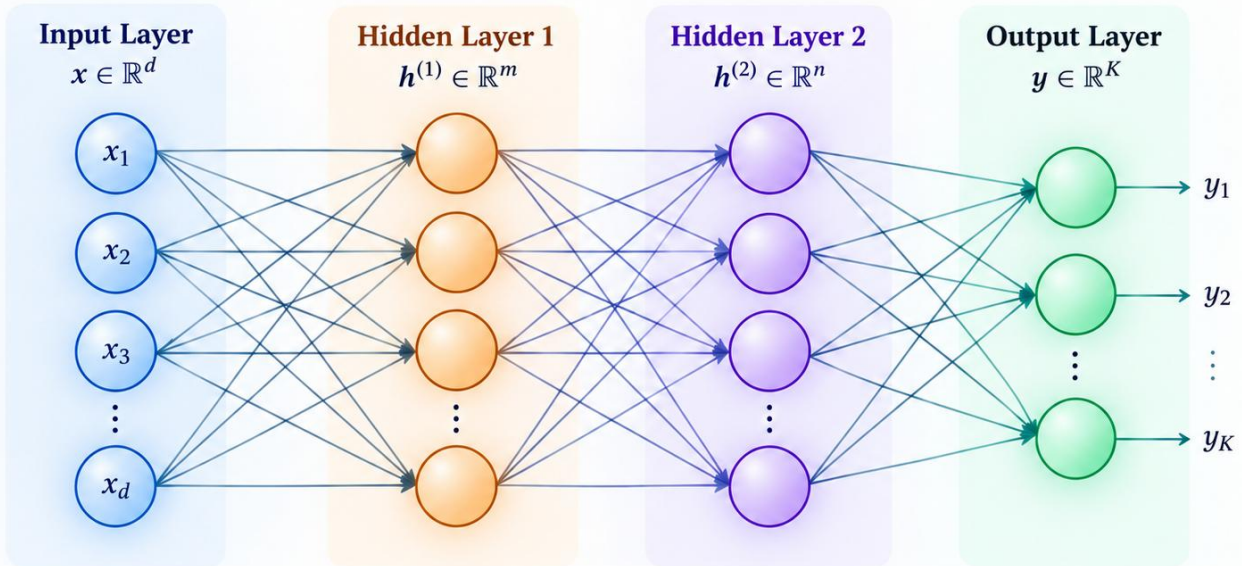
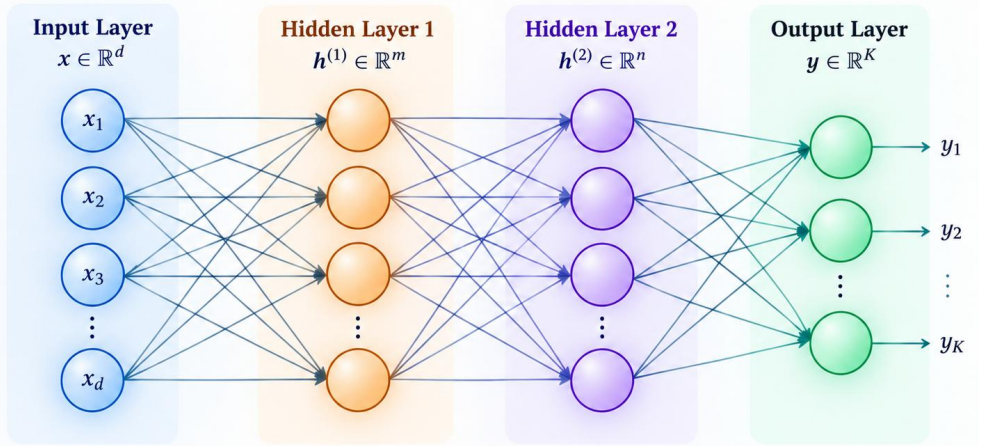
MULTILAYER PECEPTRON



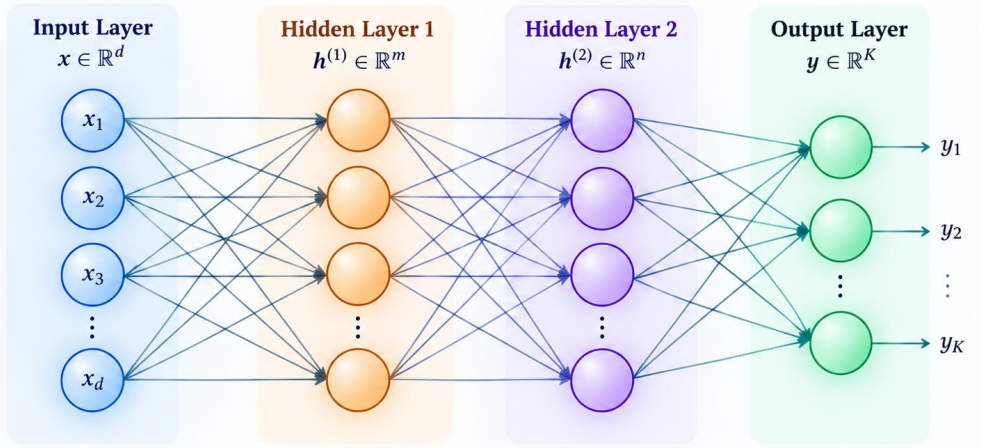
MỤC LỤC

- Multilayer Perceptron
- Forward & Backward Propagation
- Numerical Stability and Initialization
- Generalization



MỤC LỤC

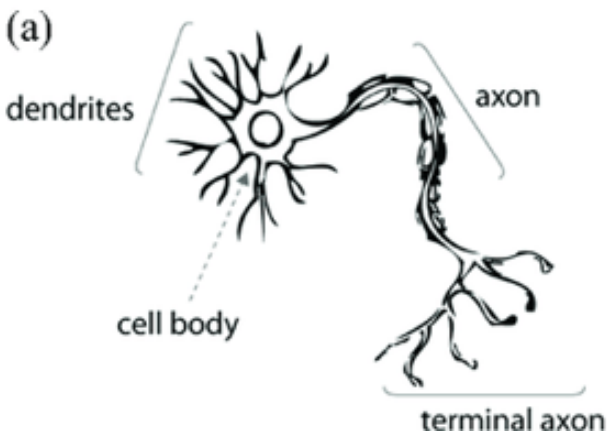
- Multilayer Perceptron
- Forward & Backward Propagation
- Numerical Stability and Initialization
- Generalization

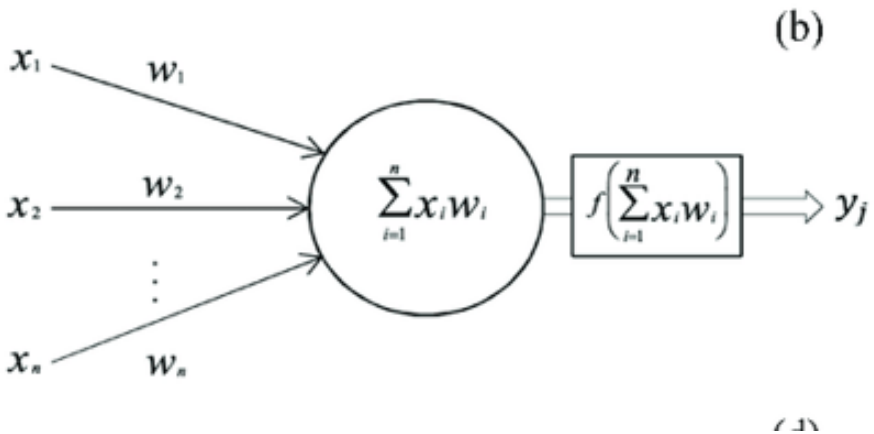


PERCEPTRON

- Perceptron (Frank Rosenblatt, 1957) là một trong những mô hình neuron nhân tạo sớm nhất.

- Cảm hứng từ cách tế bào nơ-ron hoạt động







PERCEPTRON

- Với véc-tơ đầu vào:

$$\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$$

- Ta có:

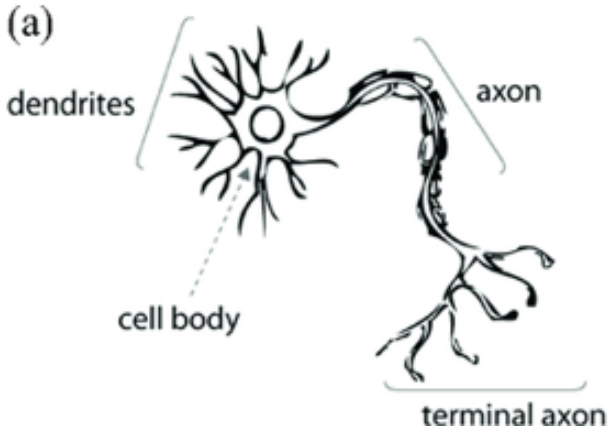
$$\boxed{z = \mathbf{w}^T\mathbf{x} + b}$$

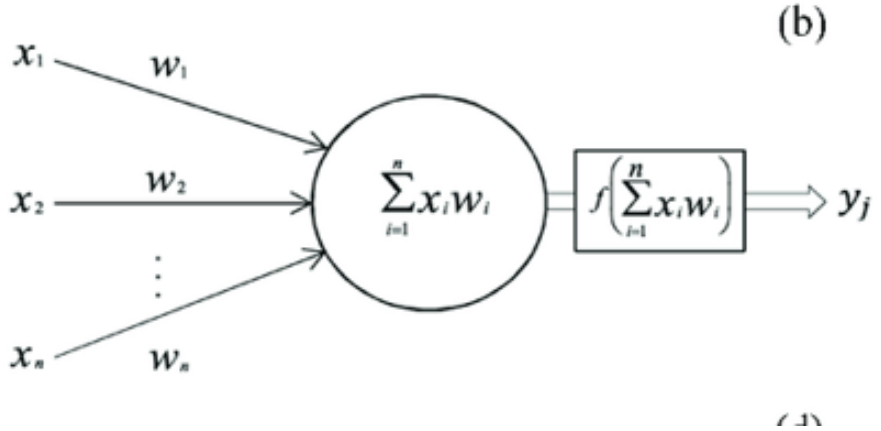
và,

$$\boxed{\hat{y} = f(z)}$$

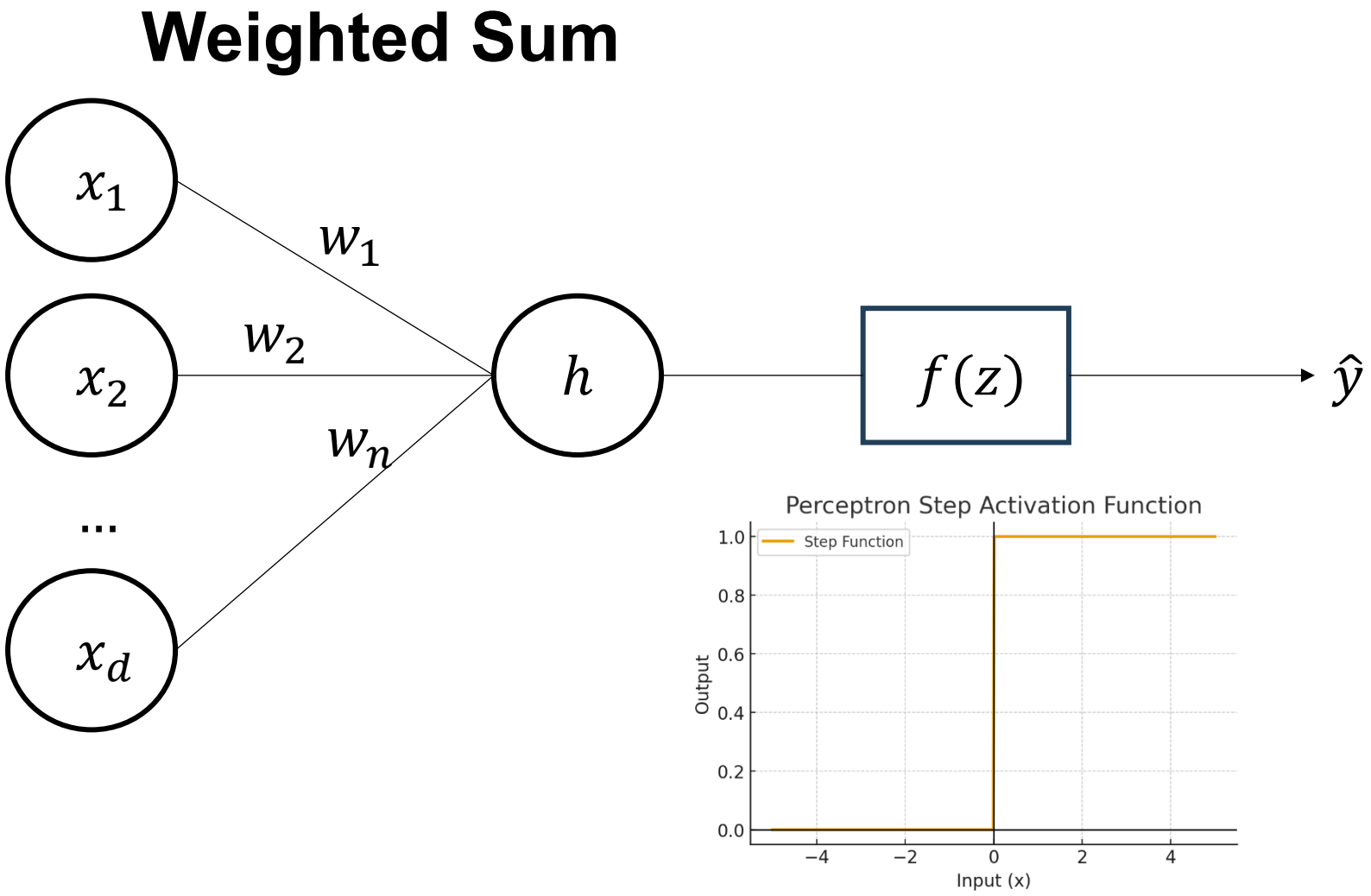
- Trong bài gốc:

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$





PERCEPTRON

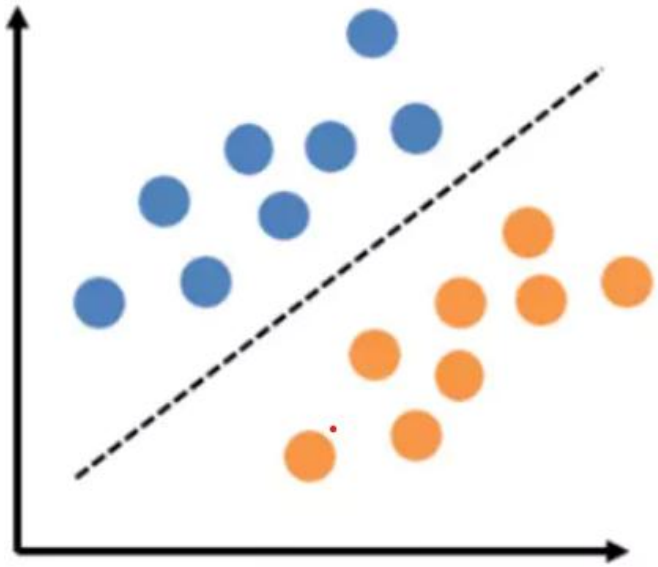

Weighted Sum
x_1
x_2
...
x_d
w_1
w_2
w_n
h
$f(z)$
$\hat{y}$
Perceptron Step Activation Function
Step Function
Output
Input (x)
1.0
0.8
0.6
0.4
0.2
0.0
−4
−2
0
2
4

PERCEPTRON

- Điều kiện phân loại:
  - Nếu $z \geq 0$ thì $\hat{y} = 1$
  - Nếu $z < 0$ thì $\hat{y} = 0$
- Có thể thấy quyết định phân loại thay đổi tại $z = 0$, ta có thể xây dựng một hyperplane:

$$\mathbf{w}^T\mathbf{x} + \mathbf{b} = 0$$

Hyperplane này gọi là Decision Boundary



VẤN ĐỀ CỦA PERCEPTRON

- Bản thân một perceptron chỉ có thể tạo ra được một decision boundary tuyến tính.

$$\mathbf{w}^T\mathbf{x} + \mathbf{b} = 0$$

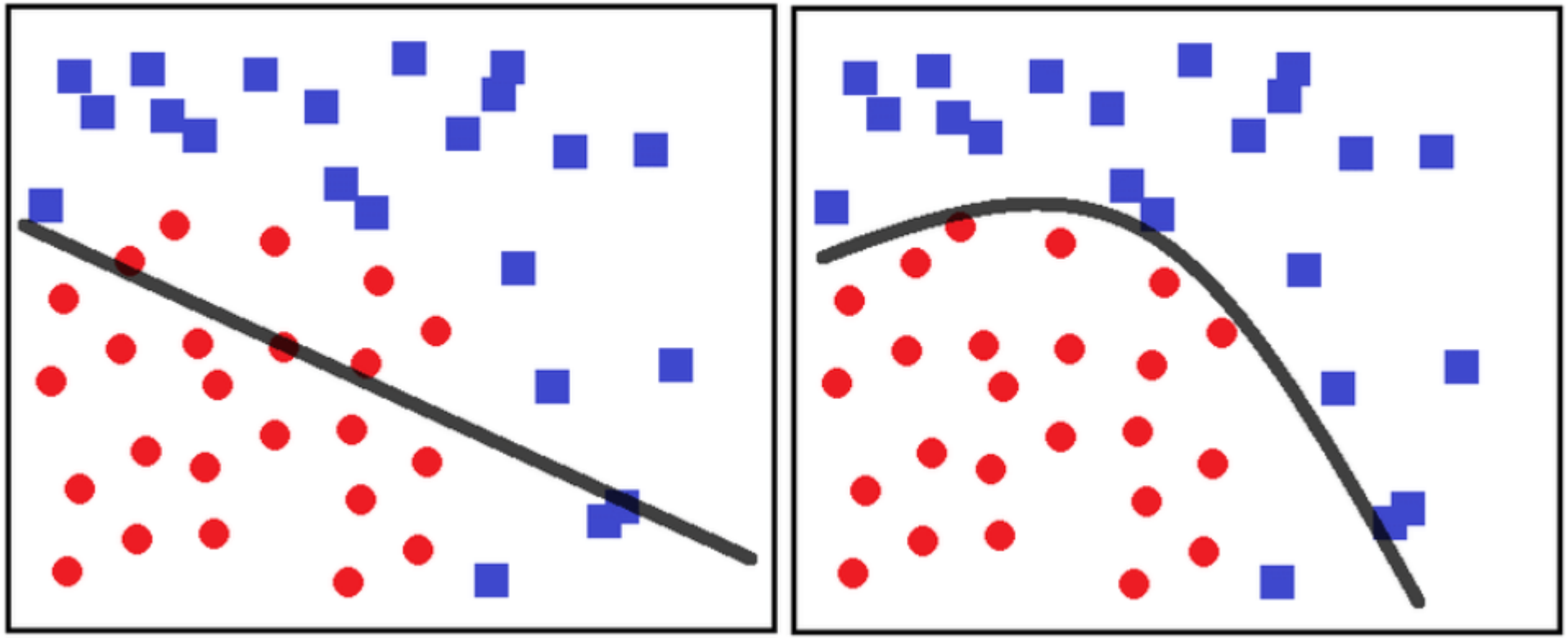
- Các hiện tượng trong thực tế có mối quan hệ phi tuyến

- VD:
- Nhiệt độ cơ thể người cao quá hay thấp quá đều nguy hiểm.
- Quan hệ của pixel trong bức ảnh.
- …



VẤN ĐỀ CỦA PERCEPTRON

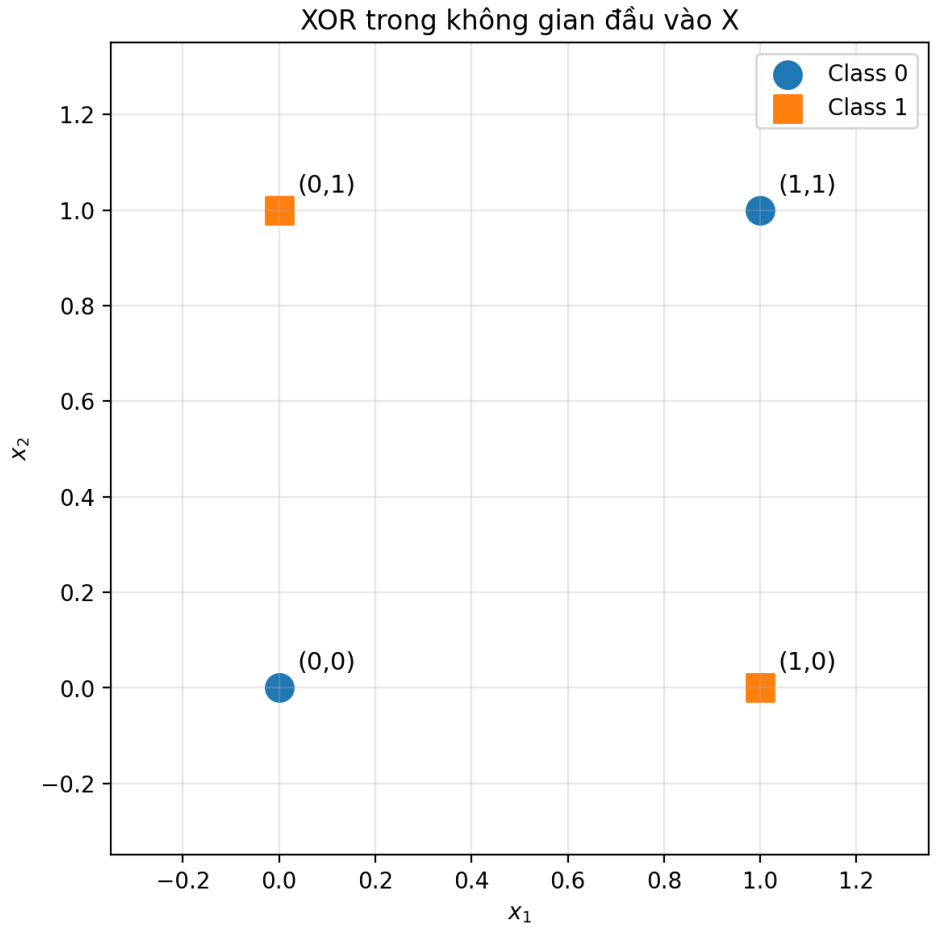
- Decision Boundary tuyến tính không đủ phân loại được các mối quan hệ thực tế.



BÀI TOÁN XOR

- Bài toán XOR

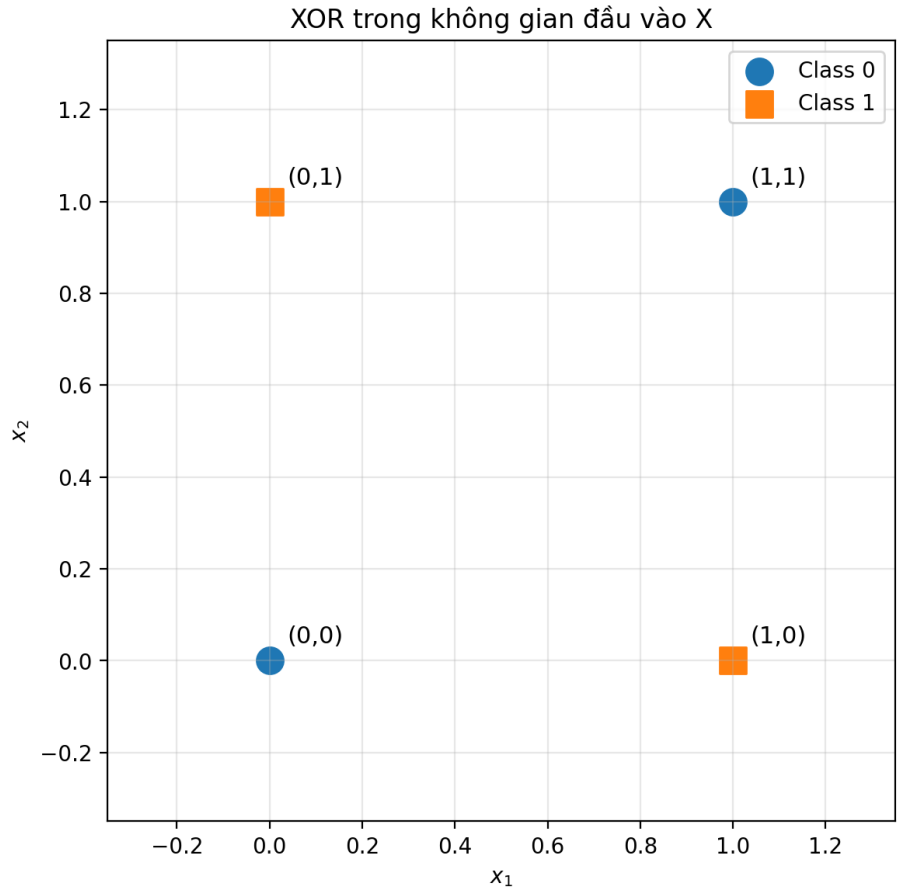
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

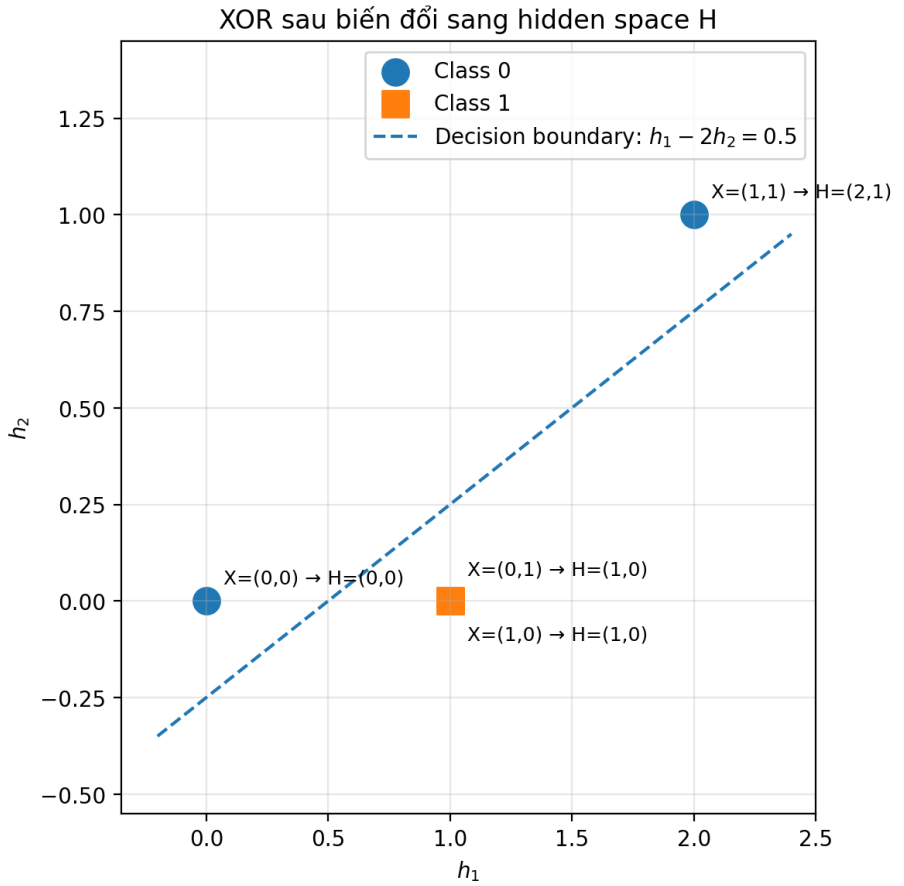


Làm sao để mô hình học được bài toán XOR?

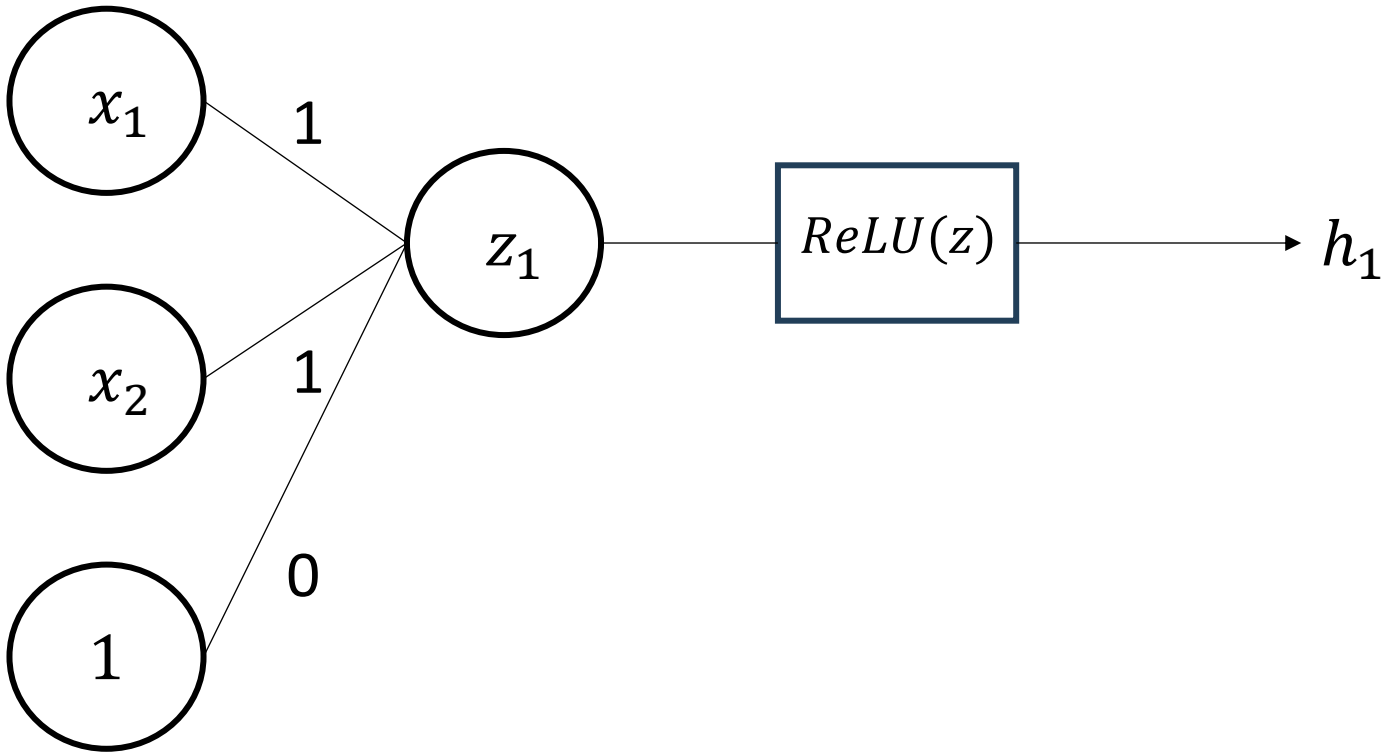
BÀI TOÁN XOR

- Trực giác: Ta biến đổi x_1 và x_2 về biểu diễn có thể phân tách tuyến tính.





BÀI TOÁN XOR

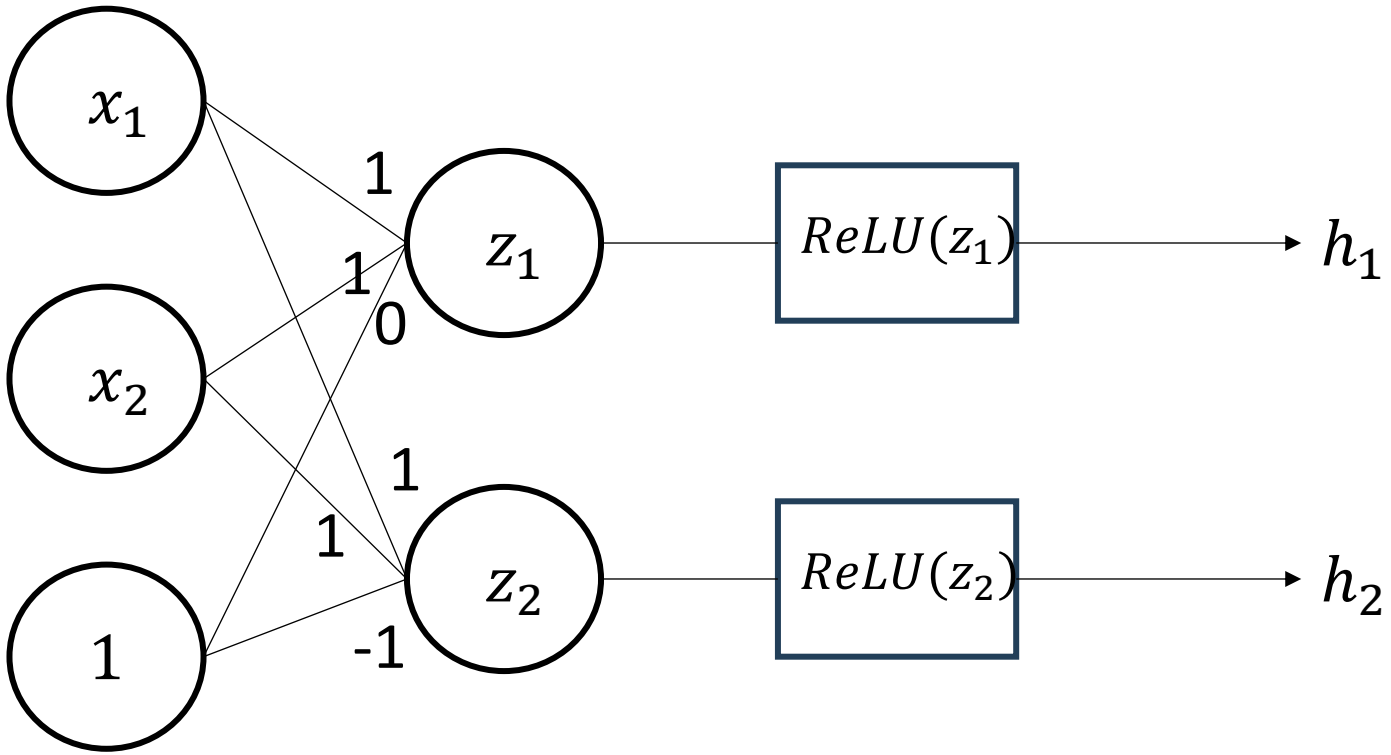


x_1	x_2	h_1
0	0	0
0	1	1
1	0	1
1	1	2

$$z_1 = x_1 + x_2$$

$$h_1 = ReLU(z_1)$$

BÀI TOÁN XOR



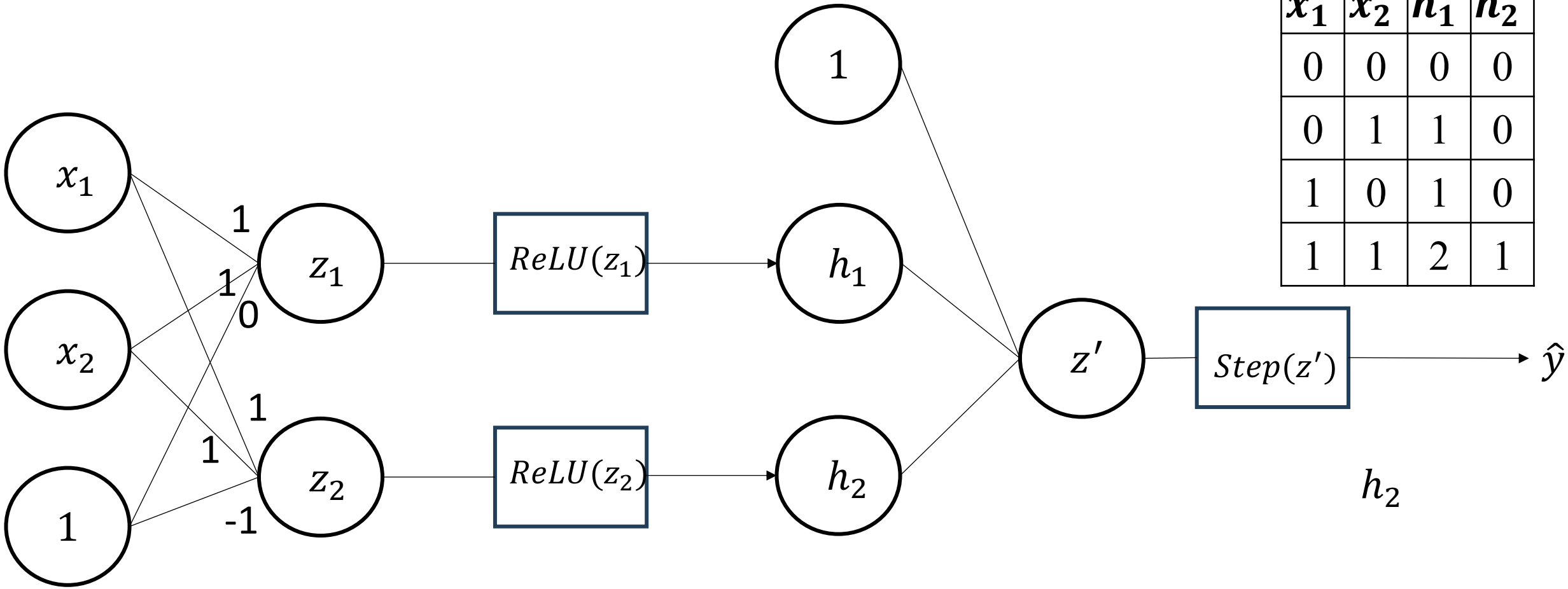
x_1	x_2	h_1	h_2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

$$z_2 = x_1 + x_2 - 1$$

$$h_2 = ReLU(z_2)$$

BÀI TOÁN XOR

x_1	x_2	h_1	h_2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1



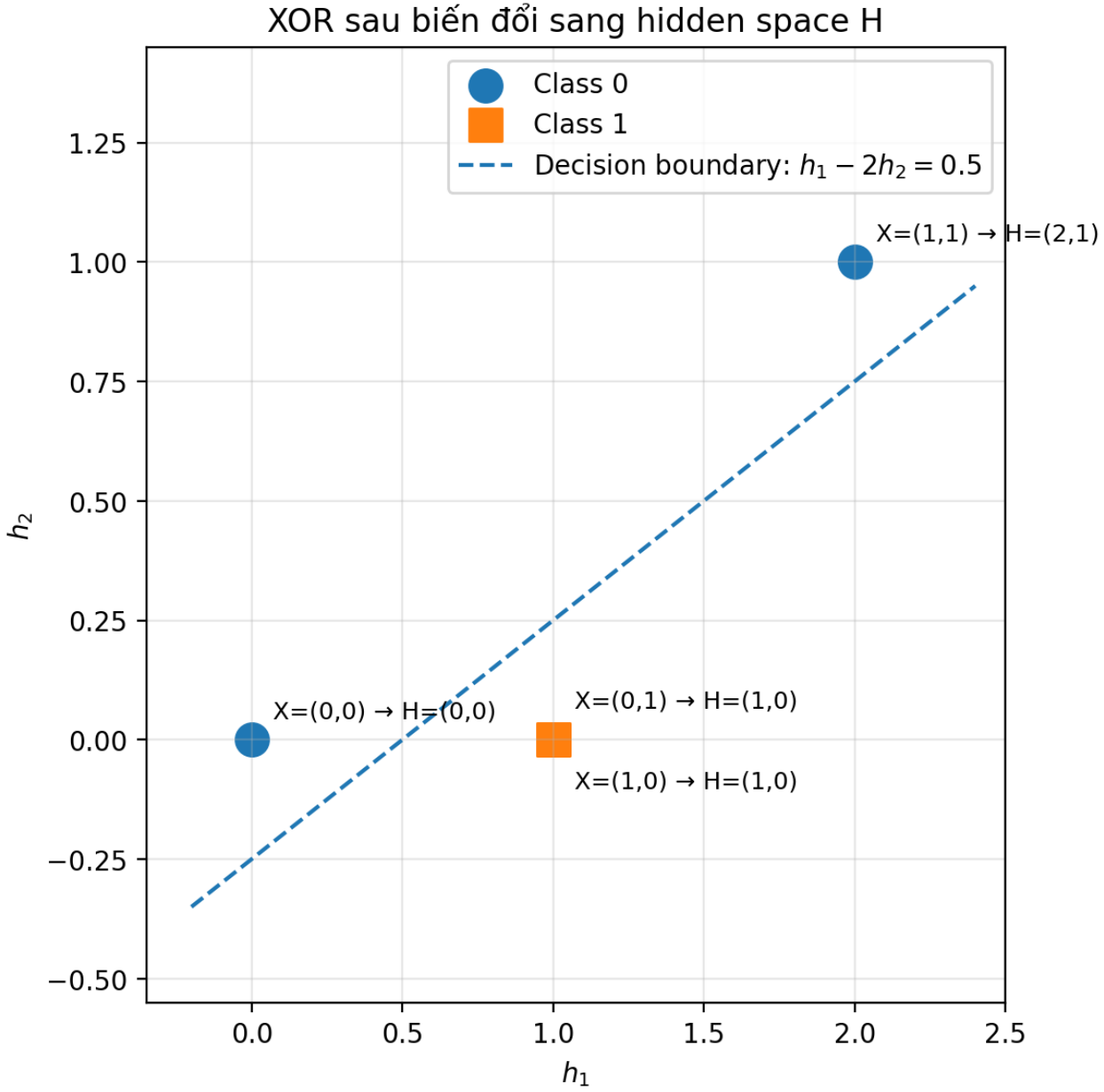
$$z' = h_1 - 2h_2 - 0.5$$

$$\hat{y} = Step(z')$$

BÀI TOÁN XOR

- Decision Boundary:

$$h_1 - 2h_2 - 0.5 = 0$$

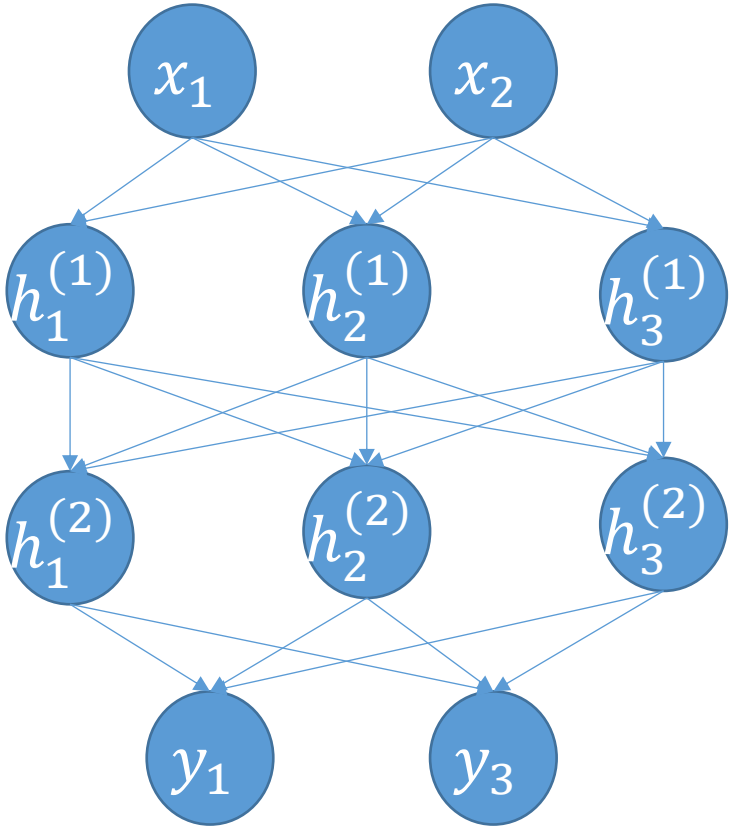


MULTILAYER PERCEPTRON

- Multilayer Perceptron là một mạng nơ-ron truyền thẳng.

$$\mathbf{X} \rightarrow \mathbf{H} \rightarrow \mathbf{O}$$

  - Trong đó:
  - $\mathbf{X}$: dữ liệu đầu vào
  - $\mathbf{H}$: lớp ẩn
  - $\mathbf{O}$: đầu ra
- Thay vì ánh xạ trực tiếp, lớp ẩn có nhiệm vụ học biểu diễn từ các đặc trưng ban đầu.
- Các lớp MLP thường được gọi là fully connected.



MULTILAYER PERCEPTRON

- Xét một minibatch đầu vào: $\mathbf{X} \in \mathbb{R}^{n \times d}$

- Hidden layer với h nơ-ron:

$$\mathbf{H} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$ ⟵ Vấn đề gì?

với $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times h}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times h}$

- Ouput layer với q đầu ra:

$$\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

với $\mathbf{W}^{(2)} \in \mathbb{R}^{h \times q}$, $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times q}$

MULTILAYER PERCEPTRON

➢ *Đặt các lớp tuyến tính lên nhau có đủ không*?

- Ta có:

$$\mathbf{H} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}, \mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

- Thay $\mathbf{H}$ vào $\mathbf{O}$:

$$\mathbf{O} = \left(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}\right)\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

$$\mathbf{O} = \mathbf{X}\mathbf{W}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

- Đặt:

$$\mathbf{W} = \mathbf{W}^{(1)}\mathbf{W}^{(2)}$$

$$\mathbf{b} = \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

- Khi đó:

$$\mathbf{O} = \mathbf{X}\mathbf{W} + \mathbf{b}$$

MULTILAYER PERCEPTRON

- Xét một minibatch đầu vào: $\mathbf{X} \in \mathbb{R}^{n \times d}$

- Hidden layer với h nơ-ron và hàm phi tuyến $\sigma(\cdot)$(Hàm kích hoạt)

$$\boxed{\mathbf{H} = \sigma\left(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}\right)}$$

với $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times h}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times h}$

- Ouput layer với q đầu ra:

$$\boxed{\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}}$$

với $\mathbf{W}^{(2)} \in \mathbb{R}^{h \times q}$, $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times q}$

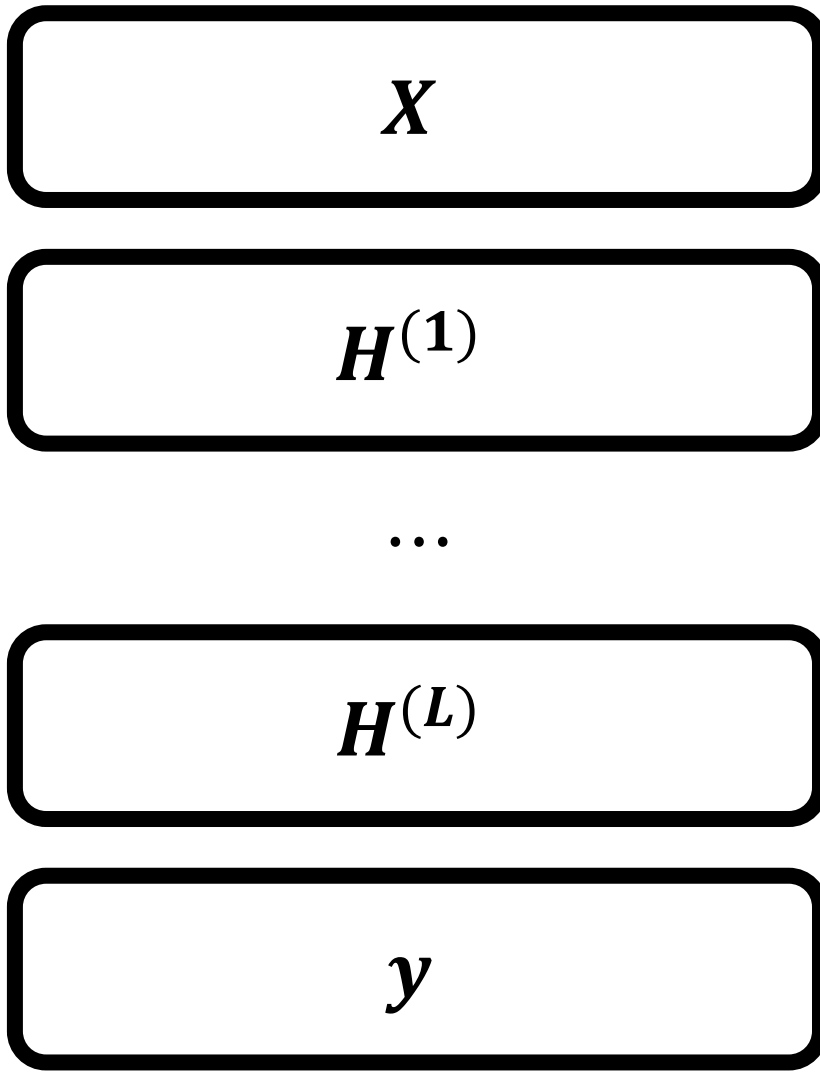
MULTILAYER PERCEPTRON

- Tiếp tục xếp chồng nhiều lớp:

$$\mathbf{H}^{(1)} = \sigma_1\left(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}\right)$$
$$\mathbf{H}^{(2)} = \sigma_2\left(\mathbf{H}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}\right)$$
$$\dots$$
$$\mathbf{O} = \mathbf{H}^{(L)}\mathbf{W}^{(L+1)} + \mathbf{b}^{(L+1)}$$

- **Deep Neural Network**

- *Tại sao lại dùng Deep Neural Network?*



UNIVERSAL APPROXIMATION THEOREMS

- Lý thuyết xấp xỉ tổng quát phát biểu:

Về nguyên tắc, các mạng nơ-ron với cấu trúc nhất định có thể xấp xỉ mọi hàm liên tục

Chứng minh: https://arxiv.org/abs/2406.10002

- Một mạng nơ-ron với kiến trúc:
  - Rộng theo chiều ngang (nhiều đơn vị nơ-ron)
  - Sâu theo chiều dọc (nhiều lớp)

Đều có khả năng biểu diễn các hàm liên tục.

- *Kiến trúc nào hiệu quả hơn?*

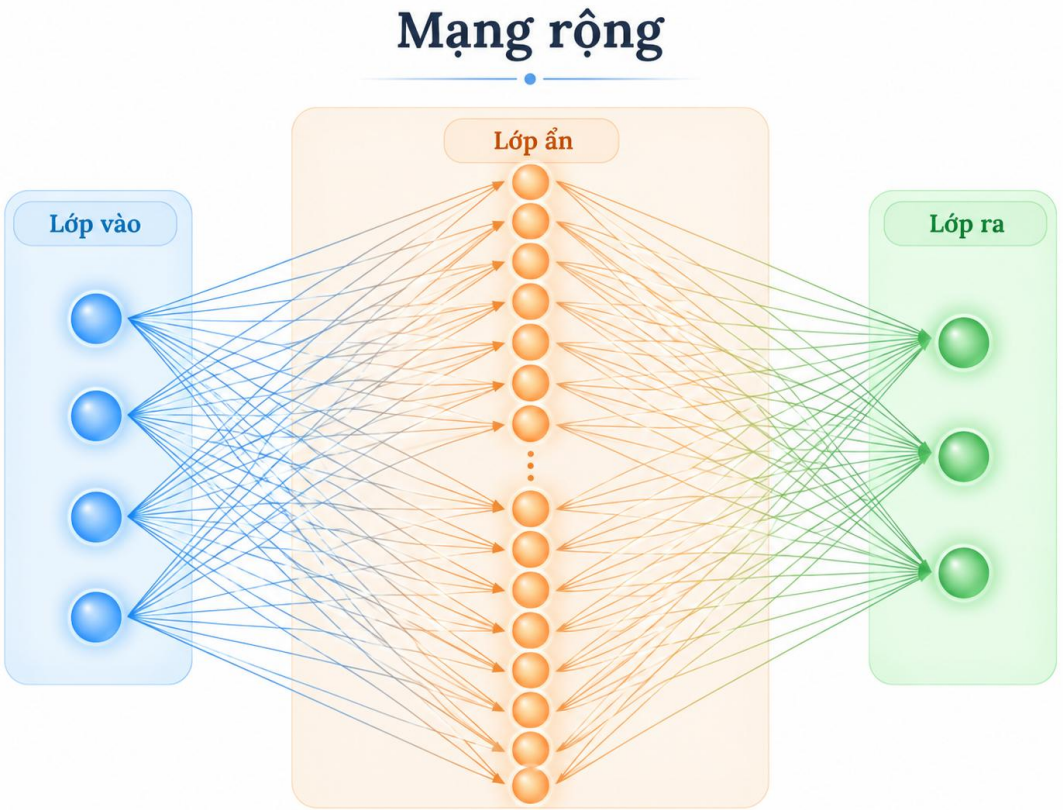
UNIVERSAL APPROXIMATION THEOREMS

➢ Trực giác của mạng rộng

- Mỗi nơ-ron trong lớp ẩn sẽ biểu diễn một mảnh đơn giản của hàm liên tục.
- Lớp đầu ra sẽ chịu trách nhiệm tổng hợp

$$f(x) \approx \sum_{i=1}^{m} \alpha_i \, \phi\left(w_i^T x + b_i\right)$$

- Khi m càng lớn, tổng thành phần này sẽ xấp xỉ được hàm liên tục.



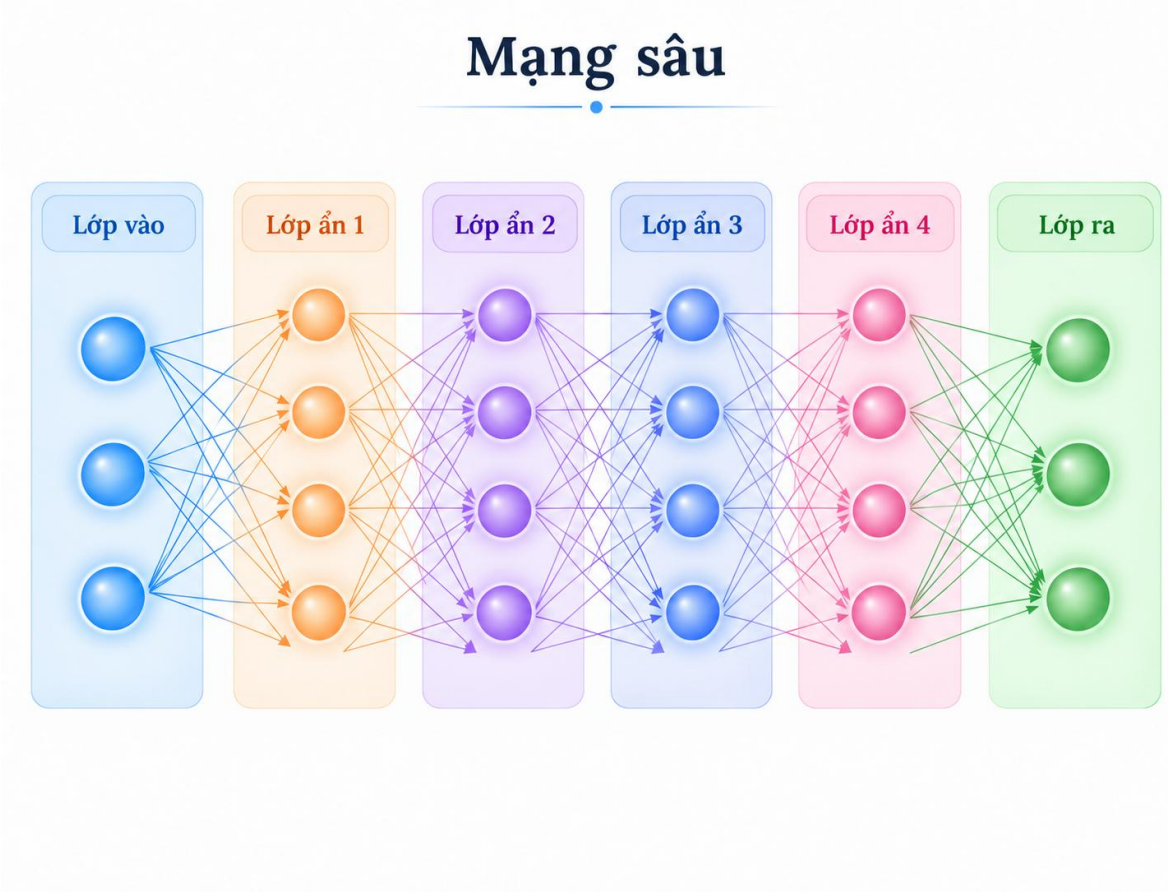
UNIVERSAL APPROXIMATION THEOREMS

- Trực giác của mạng sâu
  - Mạng học một chuỗi các biến đổi

$$f(x) = f_L\left(f_{L-1}\left(\cdots f_2\left(f_1(x)\right)\right)\right)$$

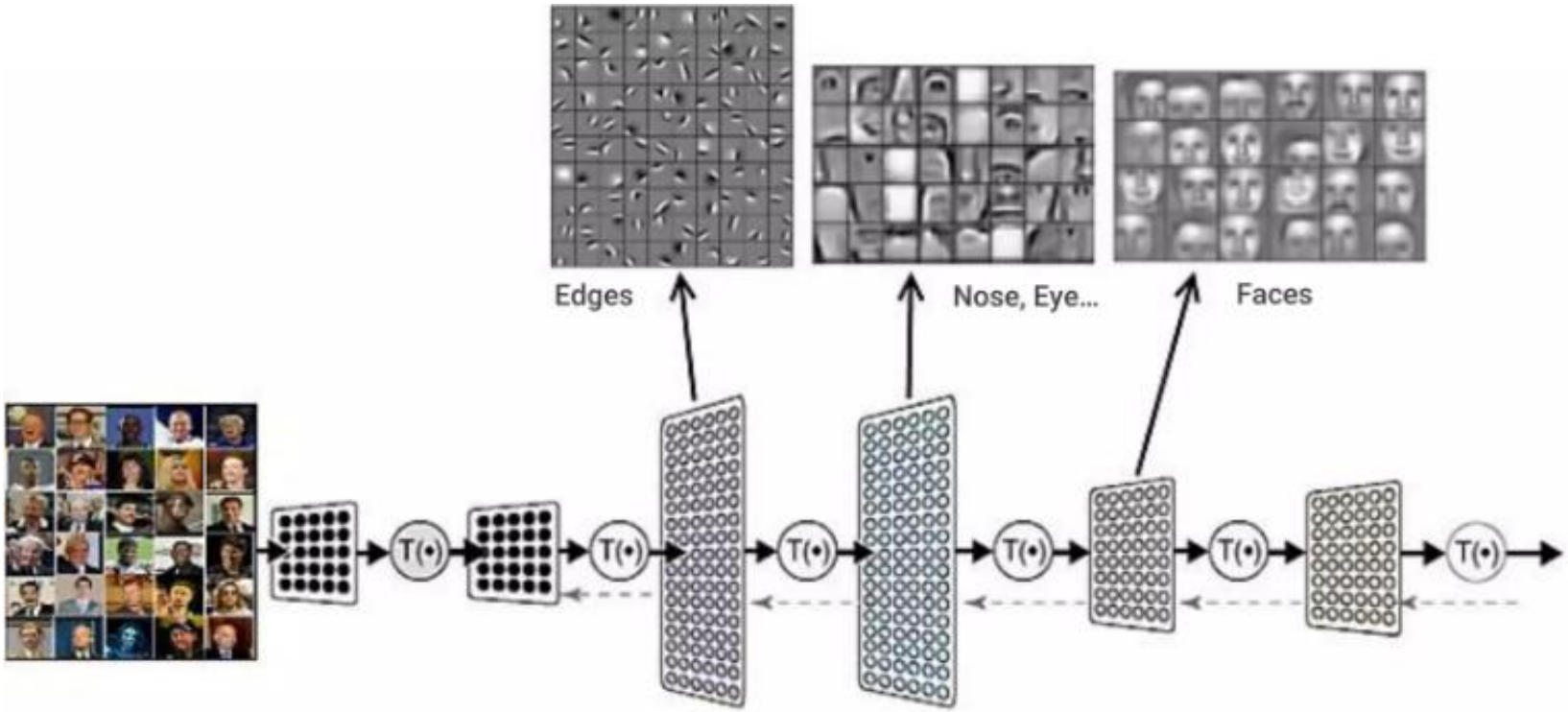
để biểu diễn xấp xỉ hàm mục tiêu

- Mạng sâu giúp biểu diễn hàm theo cấu trúc phân cấp



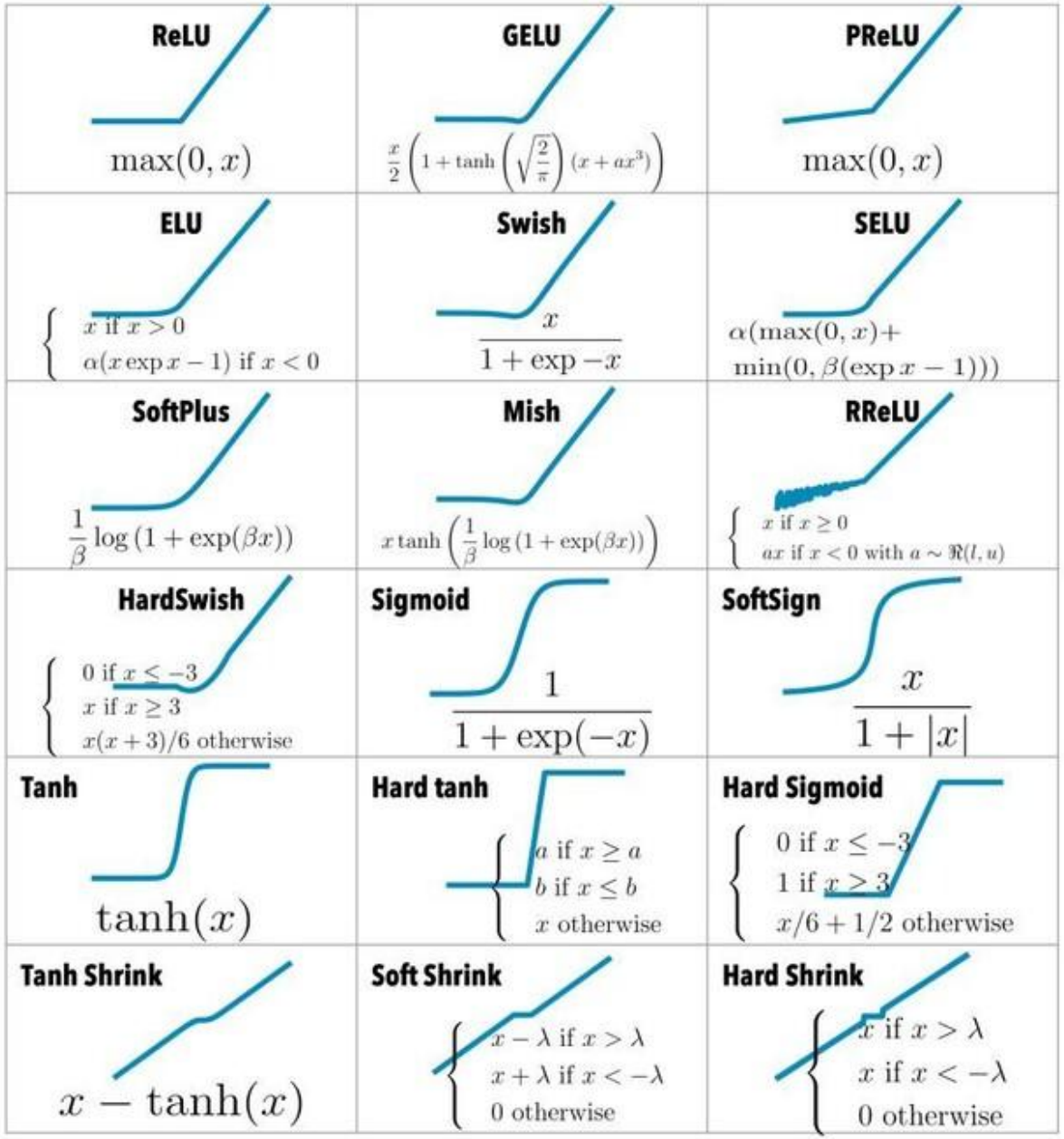
UNIVERSAL APPROXIMATION THEOREMS

- Tại sao lại sử dụng mạng sâu?
- Mạng sâu có khả năng học các biểu diễn trung gian.
- Mạng sâu có thể tái sử dụng các đặc trưng.
- Trong nhiều bài toán, mạng sâu thường có hiệu quả tham số tốt hơn so với mạng nông.



HÀM KÍCH HOẠT

Neural Network Activation Functions: a small subset!

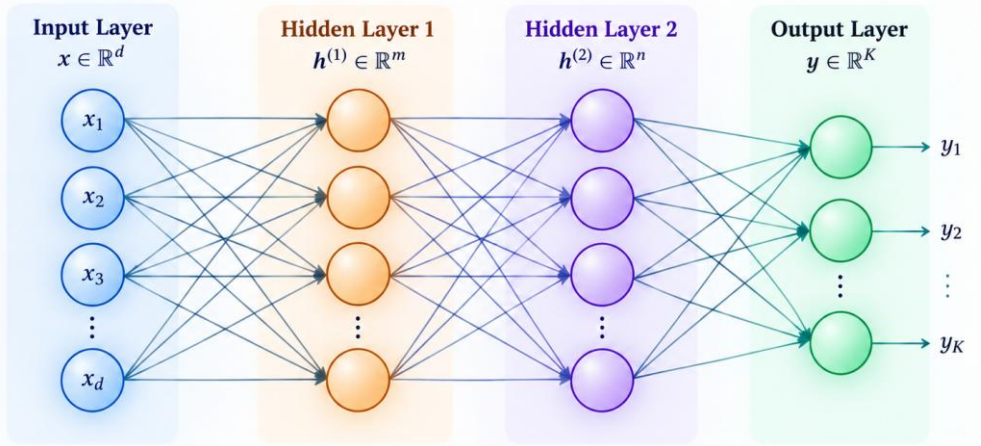


- Tạo tính phi tuyến cho mô hình

https://www.linkedin.com/pulse/activation-functions-heba-al-haddad

MỤC LỤC

- Multilayer Perceptron
- Forward & Backward Propagation
- Numerical Stability and Initialization
- Generalization

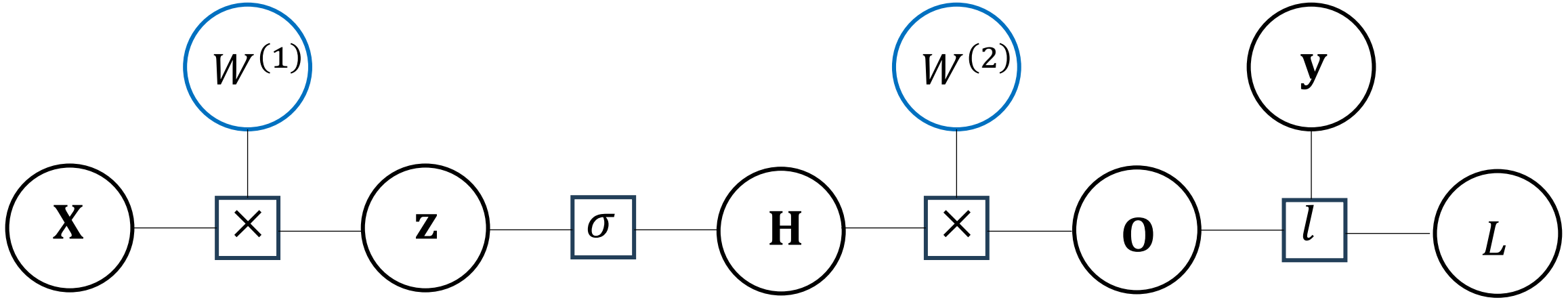


FORWARD PROPAGATION

$\mathbf{X} \in \mathbb{R}^d$

$W^{(1)} \in \mathbb{R}^{h \times d}$

$W^{(2)} \in \mathbb{R}^{q \times h}$



$\mathbf{z} = W^{(1)}\mathbf{X}$ $\mathbf{H} = \phi(\mathbf{z})$ $\mathbf{O} = W^{(2)}\mathbf{H}$ $L = l(\mathbf{o}, y)$

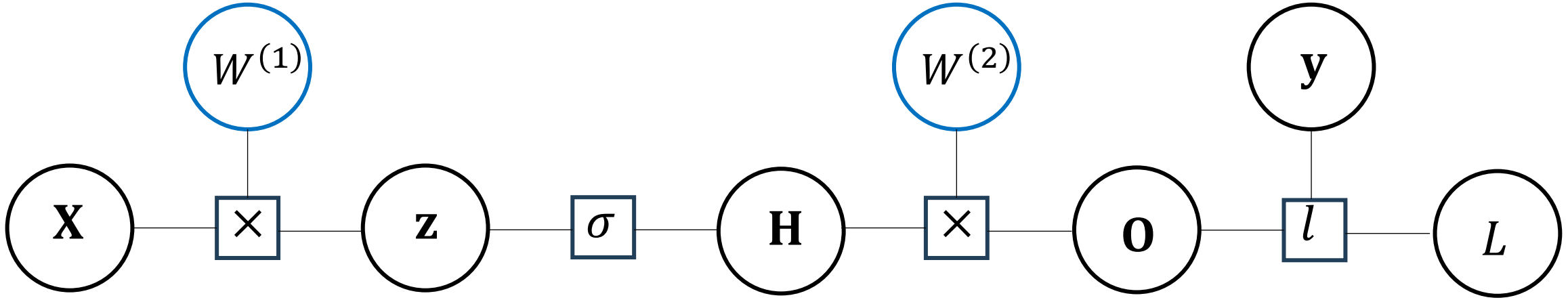


BACKPROPAGATION

$\mathbf{X} \in \mathbb{R}^d$

$W^{(1)} \in \mathbb{R}^{h \times d}$

$W^{(2)} \in \mathbb{R}^{q \times h}$



$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \mathbf{o}} \frac{\partial \mathbf{o}}{\partial W^{(2)}} = \frac{\partial L}{\partial \mathbf{o}} \mathbf{H}^T$$

$$\frac{\partial L}{\partial \mathbf{o}}$$

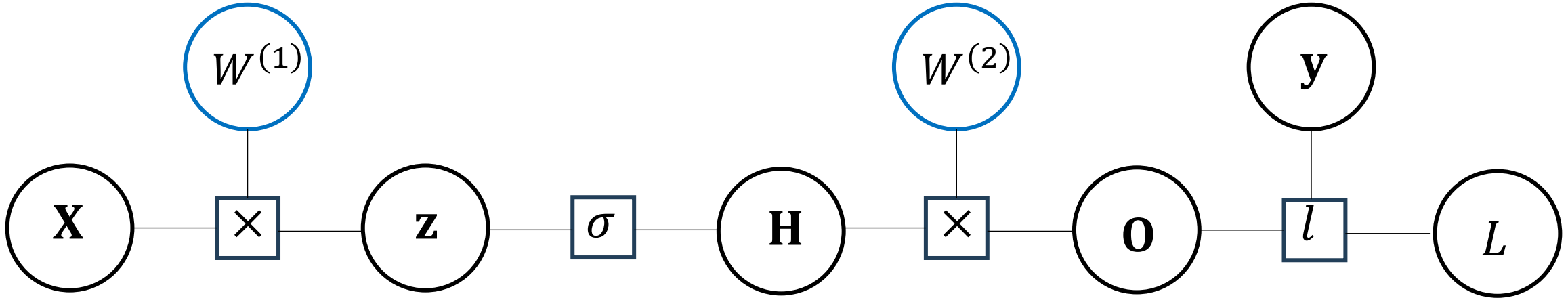
BACKPROPAGATION

$\mathbf{X} \in \mathbb{R}^d$

$W^{(1)} \in \mathbb{R}^{h \times d}$

$W^{(2)} \in \mathbb{R}^{q \times h}$

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \mathbf{o}} \mathbf{H}^T \in \mathbb{R}^{q \times h}$$



$$\frac{\partial L}{\partial \mathbf{H}} = \frac{\partial L}{\partial \mathbf{o}} \frac{\partial \mathbf{o}}{\partial \mathbf{H}} = {W^{(2)}}^T \frac{\partial L}{\partial \mathbf{o}} \in \mathbb{R}^h$$

$$\frac{\partial L}{\partial \mathbf{o}}$$

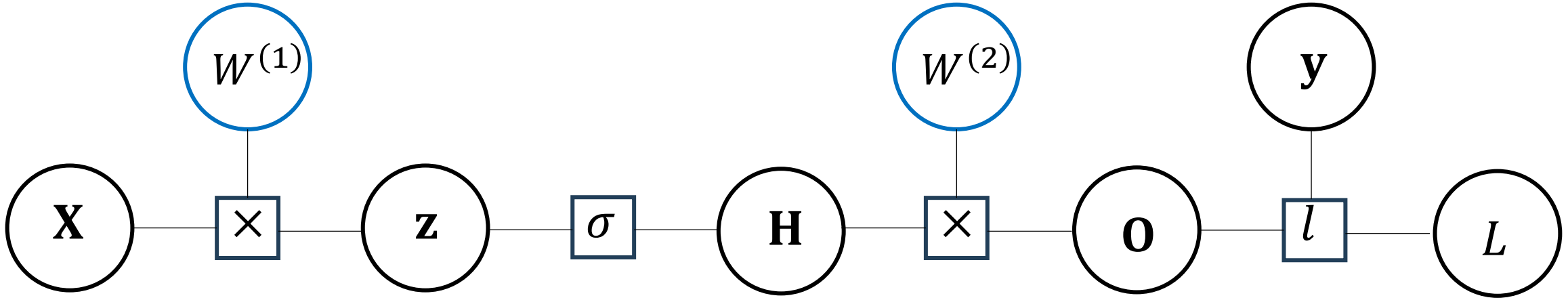
BACKPROPAGATION

$\mathbf{X} \in \mathbb{R}^d$

$W^{(1)} \in \mathbb{R}^{h \times d}$

$W^{(2)} \in \mathbb{R}^{q \times h}$

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \mathbf{o}} \mathbf{H}^T \in \mathbb{R}^{q \times h}$$



$$\frac{\partial L}{\partial \mathbf{H}} W^{(2)^T} \frac{\partial L}{\partial \mathbf{o}} \in \mathbb{R}^h$$

$$\frac{\partial L}{\partial \mathbf{o}}$$

$$\frac{\partial L}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{H}} \frac{\partial \mathbf{H}}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{H}} \odot \phi'(\mathbf{z}) \in \mathbb{R}^h$$

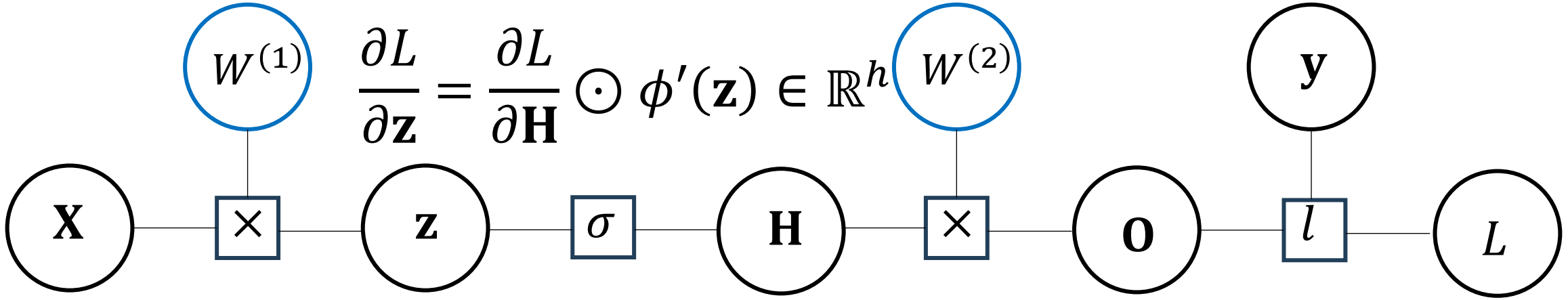
BACKPROPAGATION

$\mathbf{X} \in \mathbb{R}^d$

$W^{(1)} \in \mathbb{R}^{h \times d}$

$W^{(2)} \in \mathbb{R}^{q \times h}$

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \mathbf{o}} \mathbf{H}^T \in \mathbb{R}^{q \times h}$$



$$\frac{\partial L}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{H}} \odot \phi'(\mathbf{z}) \in \mathbb{R}^h$$

$$\frac{\partial L}{\partial \mathbf{H}} W^{(2)^T} \frac{\partial L}{\partial \mathbf{o}} \in \mathbb{R}^h$$

$$\frac{\partial L}{\partial \mathbf{o}}$$

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial W^{(1)}} = \frac{\partial L}{\partial \mathbf{z}} \mathbf{x}^T \in \mathbb{R}^{h \times d}$$

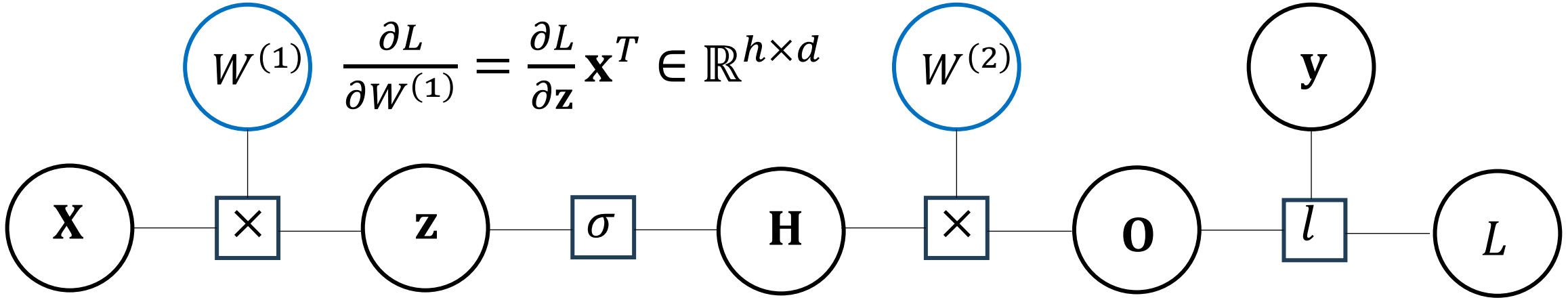
BACKPROPAGATION

$\mathbf{X} \in \mathbb{R}^d$

$W^{(1)} \in \mathbb{R}^{h \times d}$

$W^{(2)} \in \mathbb{R}^{q \times h}$

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \mathbf{o}} \mathbf{H}^T \in \mathbb{R}^{q \times h}$$



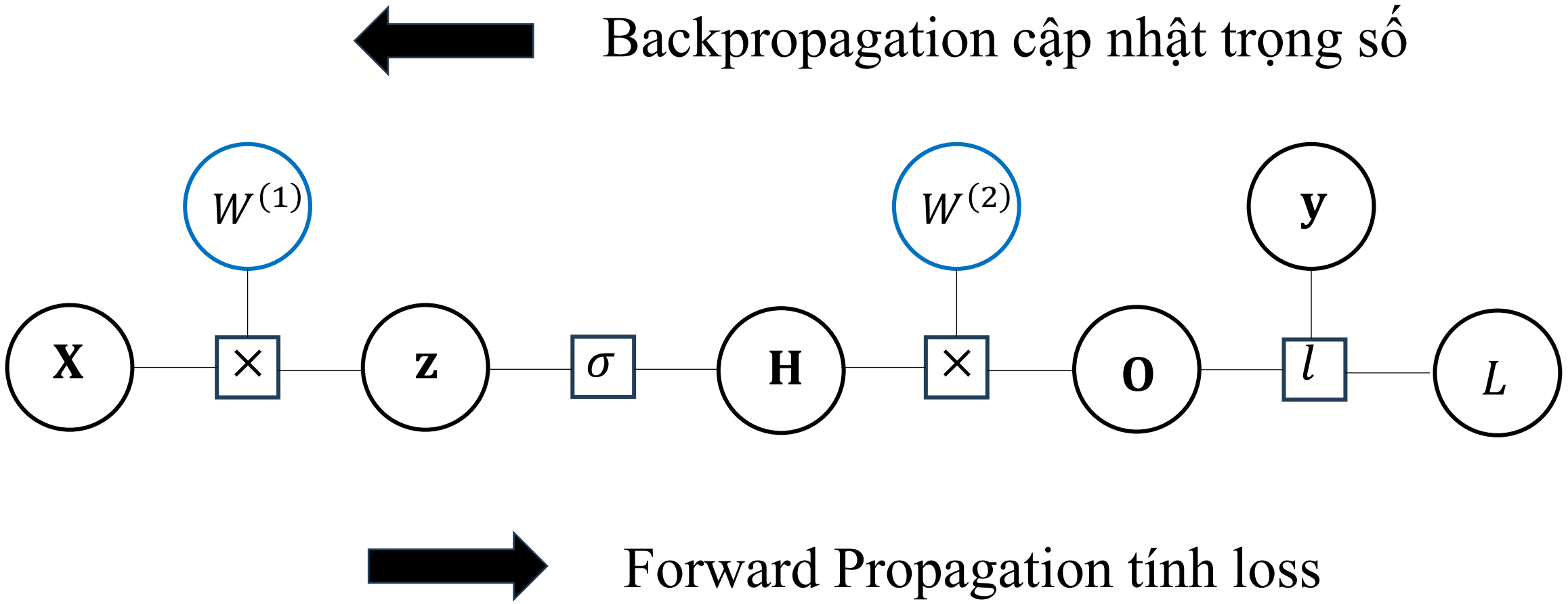
$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \mathbf{z}} \mathbf{x}^T \in \mathbb{R}^{h \times d}$$

$$\frac{\partial L}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{H}} \odot \phi'(\mathbf{z}) \in \mathbb{R}^h$$

$$\frac{\partial L}{\partial \mathbf{H}} W^{(2)^T} \frac{\partial L}{\partial \mathbf{o}} \in \mathbb{R}^h$$

$$\frac{\partial L}{\partial \mathbf{o}}$$

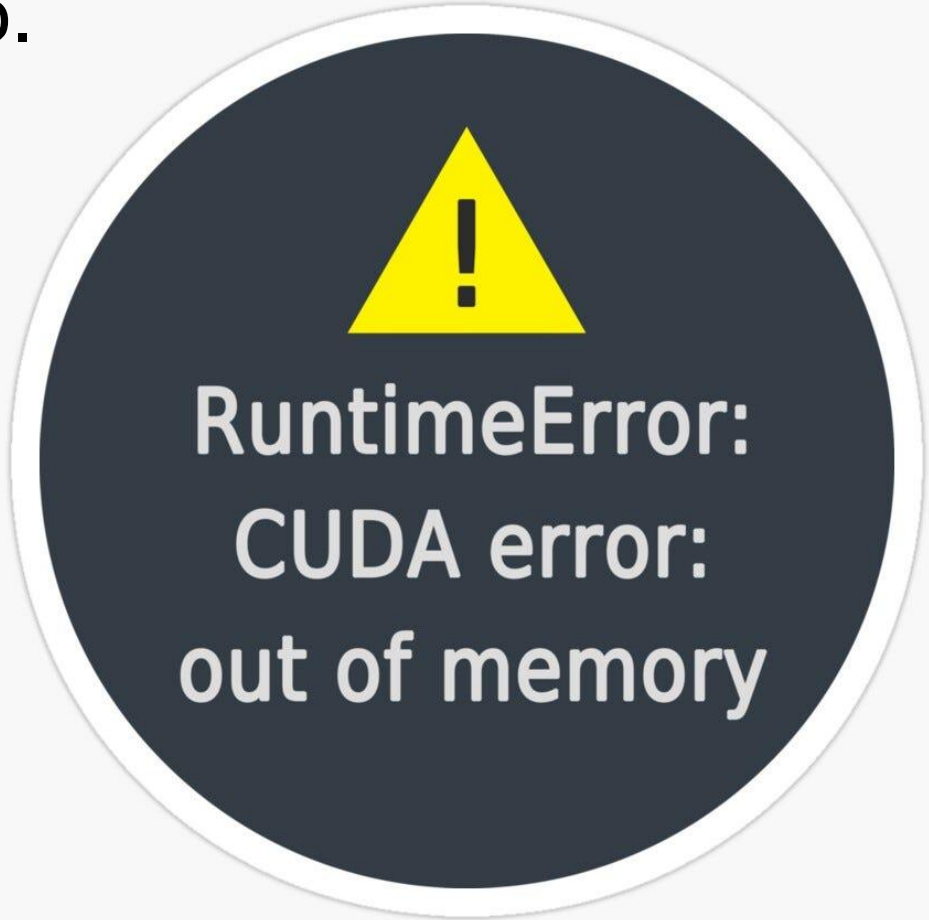
TRAINING LOOP



BỘ NHỚ TRONG QUÁ TRÌNH TRAINING

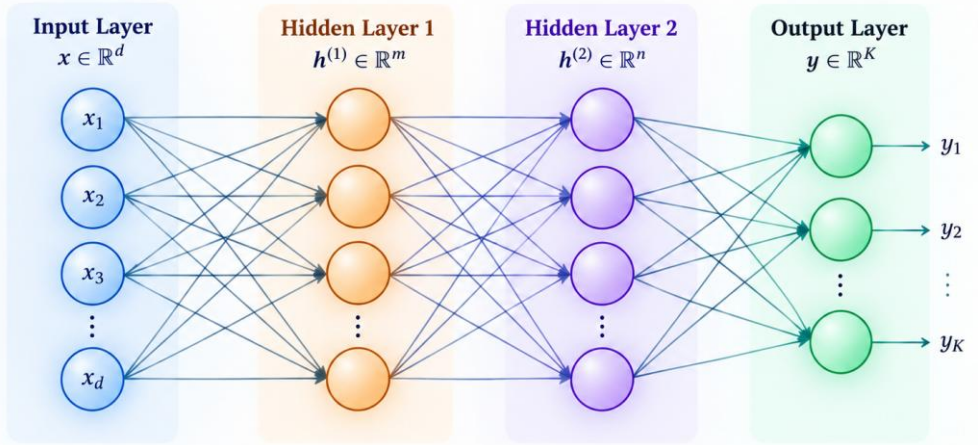
- Quá trình inference có thể giải phóng các giá trị trung gian không cần thiết.
- Trong training, quá trình backpropagation cần lưu lại các giá trị trung gian để phục vụ việc cập nhật trọng số.

- Bộ nhớ có thể tăng theo:
  - Kích thước batch;
  - Độ sâu của mạng;
  - Số lượng nơ-ron mỗi lớp.



MỤC LỤC

- Multilayer Perceptron
- Forward & Backward Propagation
- Numerical Stability and Initialization
- Generalization



GRADIENT TRONG MẠNG NƠ-RON SÂU

- Với một nơ-ron mạng sâu có L lớp ẩn, gradient tạo lớp ẩn (l) như sau:

$$\frac{\partial o}{\partial W^{(l)}} = \frac{\partial o}{\partial h^{(L)}} \frac{\partial h^{(L)}}{\partial h^{(L-1)}} \cdots \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \frac{\partial h^{(l)}}{\partial W^{(l)}}$$

- Có 2 trường hợp xảy ra:
- Nếu gradient sau mỗi lớp có xu hướng nhỏ đi -> có thể **Vanishing Gradient.**
- Nếu gradient sau mỗi lớp có xu hướng tang dần -> có thể **Exploding Gradient.**

VANISHING GRADIENT

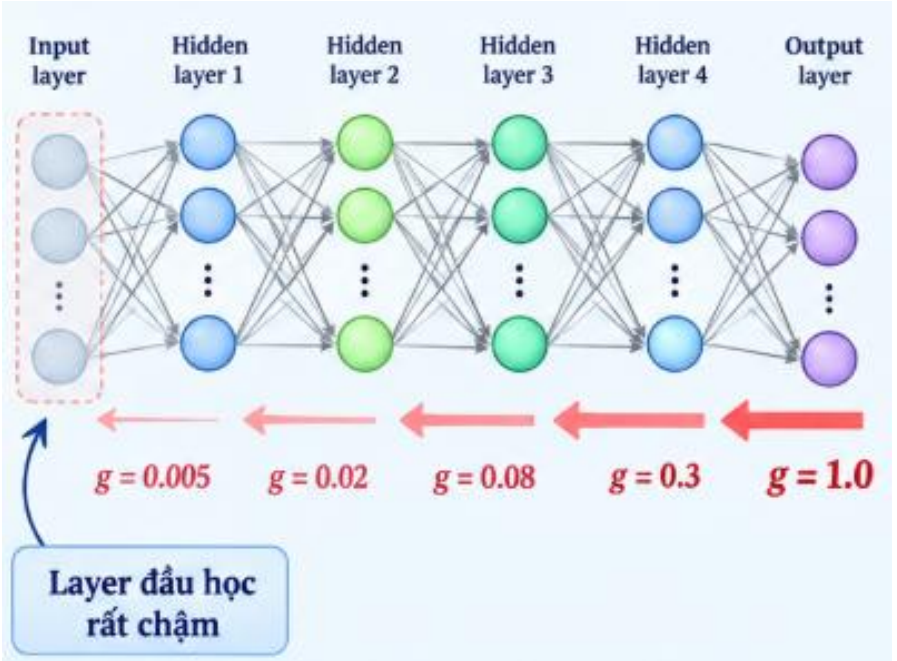
- Vanishing gradient xảy ra khi:

$$\left|\frac{\partial L}{\partial W}\right| \to 0$$

$\Rightarrow$ Không thể cập nhật trọng số:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

- Hệ quả là mô hình hội tụ chậm hoặc không thể hội tụ.



EXPLODING GRADIENT

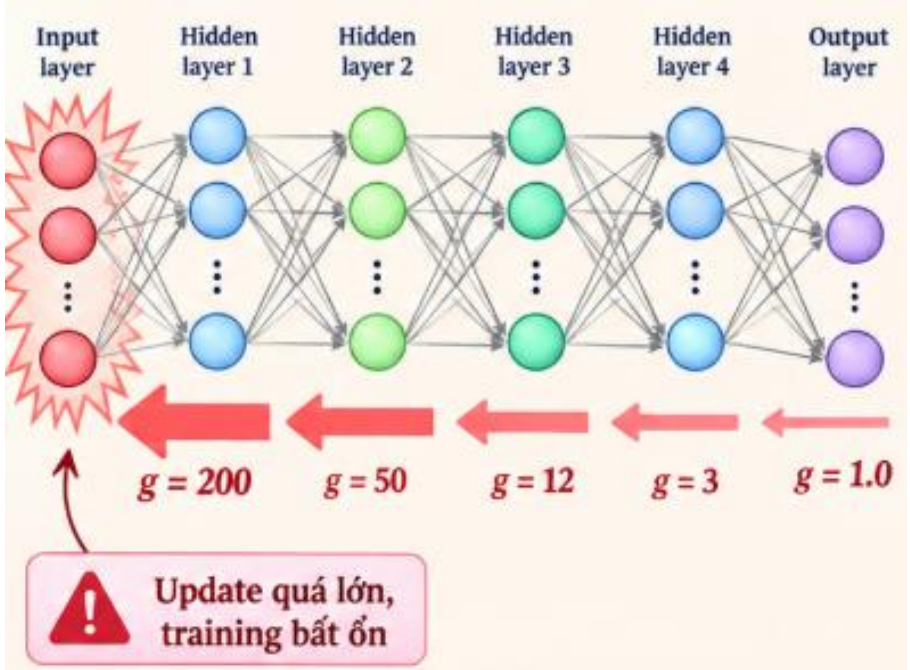
- Vanishing gradient xảy ra khi:

$$\left|\frac{\partial L}{\partial W}\right| \to \infty$$

$\Rightarrow$ Không thể cập nhật trọng số:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

- Hệ quả:
- Loss dao động mạnh,mô hình khó hội tụ.
- Có thể bị Overflow



KHỞI TẠO THAM SỐ

- Trước khi bắt đầu học, mô hình cần được khởi tạo với một bộ tham số ban đầu.
  - *Phải khởi tạo như thế nào?*

- Khởi tạo tất cả các trọng số ẩn cùng một giá trị

$\Rightarrow$ Gây ra hiện tượng symmetry cho mô hình nơ-ron

- Cần khởi tạo các trọng số một cách ngẫu nhiên (Random Initialization)

$$W_{ij} \sim \text{Random Distribution}$$

KHỞI TẠO THAM SỐ

- Xét một nơ-ron tuyến tính:

$$o_i = \sum_{j=1}^{n_{in}} w_{ij}\, x_j$$

- Giả sử $E[w_{ij}] = 0$.
- Khi đó:

$$E[o_i] = E\left[\sum_{j=1}^{n_{in}} w_{ij}\, x_j\right]$$

$$= \sum_{j=1}^{n_{in}} E\left[w_{ij} x_j\right]$$

$$= E[w_{ij}]E[x_j] = 0$$

KHỞI TẠO THAM SỐ

- Xét một nơ-ron tuyến tính:

$$o_i = \sum_{j=1}^{n_{in}} w_{ij}\, x_j$$

- Với $Var(o_i)$

$$\begin{aligned} Var(o_i) &= E[{o_i}^2] - (E[o_i])^2 \\ &= \sum_{j=1}^{n_{in}} E\left[{w_{ij}}^2 {x_j}^2\right] - 0 \\ &= \sum_{j=1}^{n_{in}} E\left[{w_{ij}}^2\right] E\left[{x_j}^2\right] = n_{in}\sigma_w^2\sigma_x^2 \end{aligned}$$

KHỞI TẠO THAM SỐ

- Mục tiêu muốn tín hiệu không tăng hay giảm khi truyền quả các lớp

$$Var(o_i) \approx Var(x_j)$$

- Hay:

$$n_{in}\sigma_w^2\sigma_x^2 \approx \sigma_x^2$$

- Chia cả 2 vế cho σ_x^2:

$$n_{in}\sigma_w^2 \approx 1$$

$$\sigma_w^2 \approx \frac{1}{n_{in}}$$

KHỞI TẠO THAM SỐ

- Forward propagation cần

$$n_{in}\sigma_w^2 \approx 1$$

- Ta cũng muốn backpropagation ổn định

$$n_{out}\sigma_w^2 \approx 1$$

- Nếu

$$n_{in} \neq n_{out}$$

Thì ta không thể thỏa mãn được cả hai điều kiện bằng cùng một σ_w^2

KHỞI TẠO XAVIER

- Khởi tạo Xaiver muốn ổn định cả quá trình forward và backward của mạng.

$$\frac{1}{2}(n_{out} + n_{in})\sigma_w^2 \approx 1$$

- Khởi tạo Xavier lấy weight từ Gaussian

$$W_{ij} \sim N\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

- Hoặc từ uniform

$$W \sim U(-a, a)$$

  - Với $a = \sqrt{\frac{6}{n_{in}+n_{out}}}$

TỔNG KẾT

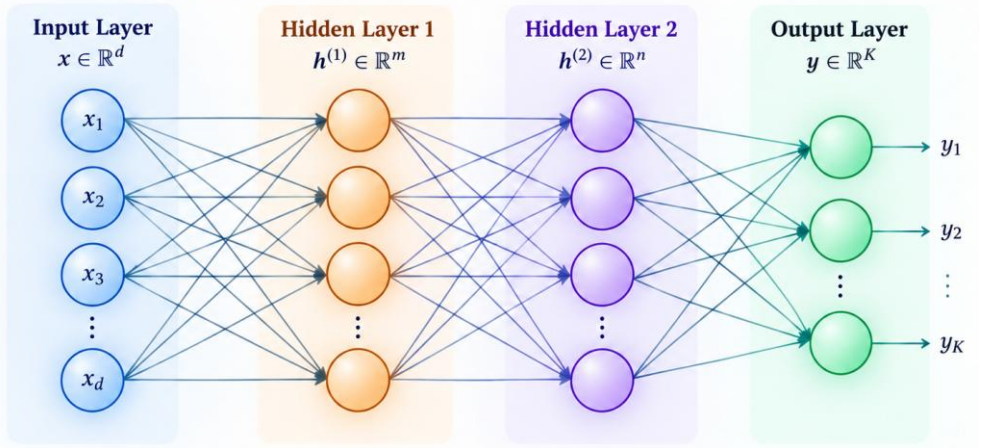
- Khởi tạo có 2 nhiệm vụ chính
  - Phá tính symmetri để các nơ-ron học các đặc trưng khác nhau.

  - Kiểm soát $Var(W)$ để đảm bảo:
  - Forward không explode/vanish
  - Gradient không explode/vanish

- Việc ổn định số học là đặc biệt quan trọng khi độ sau mạng nơ-ron tang.

MỤC LỤC

- Multilayer Perceptron
- Forward & Backward Propagation
- Numerical Stability and Initialization
- Generalization



GÓC NHÌN BIAS-VARIANCE

- Ta có:

$$\text{Expected Error} = Bias^2 + Variance + Noise$$

- Mô hình đơn giản:

$$Bias \uparrow, \quad Variance \downarrow$$

- Mô hình phức tạp:

$$Bias \downarrow, \quad Variance \uparrow$$

- Trực giác: tăng complexity ban đầu giúp mô hình tốt hơn, nhưng quá nhiều complexity thì sẽ gây ra overfitting.

NGHỊCH LÝ TRONG HỌC SÂU

- Mô hình học sâu hiện đại thường có:

$$\text{Số tham số} \gg \text{Số mẫu huấn luyện}$$

- Khi đó có thể tồn tại rất nhiều nghiệm để:

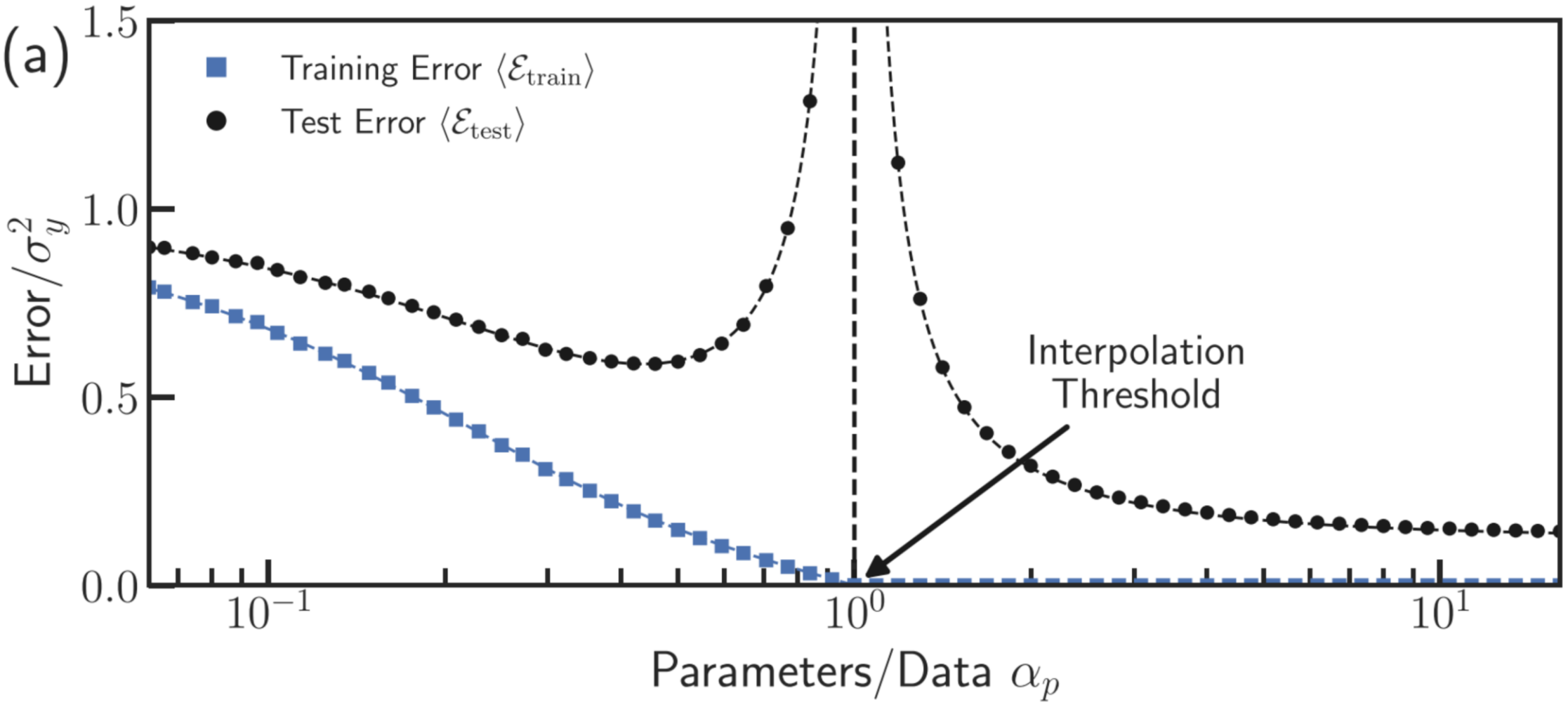
$$f_{W_j}(x_i) = y_i$$

Với mấu huấn luyện.

- *Tại sao mô hình vẫn chọn được nghiệm tổng quát hóa tốt?*

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 XL	1.3B	24	2048	24	128
GPT-3 2.7B	2.7B	32	2560	32	80
GPT-3 6.7B	6.7B	32	4096	32	128
GPT-3 13B	13.0B	40	5140	40	128
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128

DOUBLE DESCENT

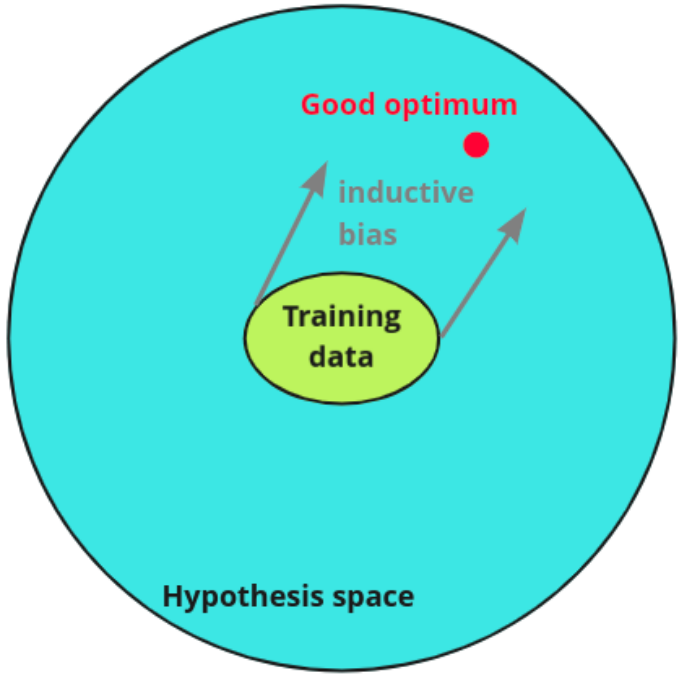


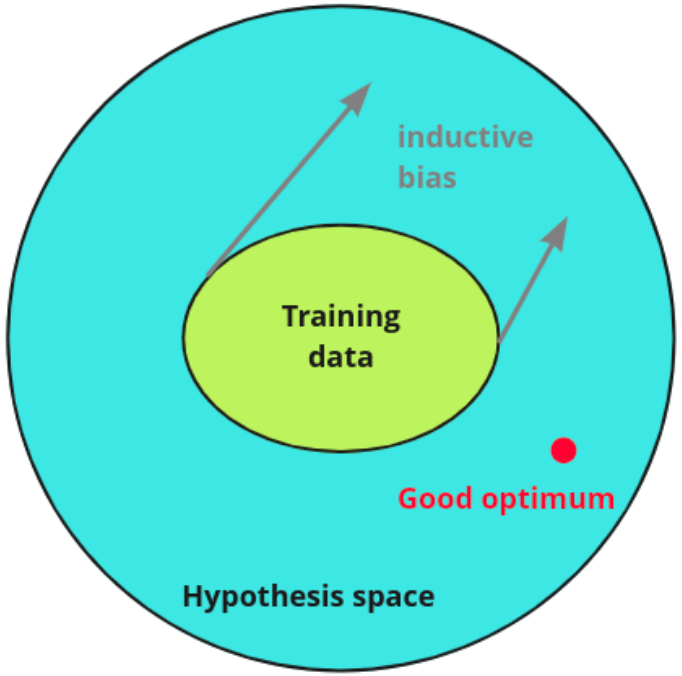
INDUCTIVE BIAS

- Inductive Bias là tập hợp các giả định mà mô hình sử dụng để dự đoán kết quả cho dữ liệu chưa thấy..

- Có thể hiểu:

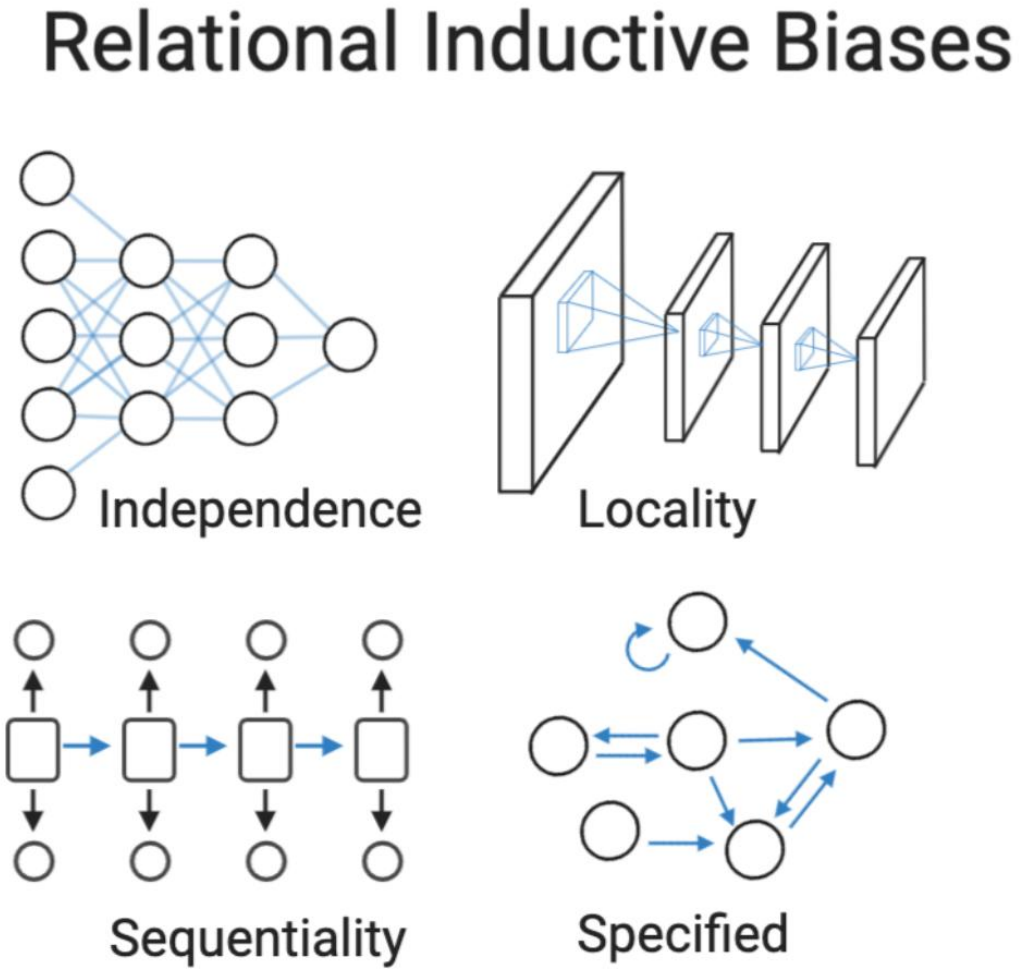
$$\text{Training Data} + \text{Inductive Bias} \rightarrow \text{Learned Solution}$$





INDUCTIVE BIAS

- Trong mô hình học sâu, inductive bias có thể đến từ:
  - Network Architecture
  - Initialization
  - Optimizer
  - Loss Function
  - Regularization
  - Data Representaion/Agumentation

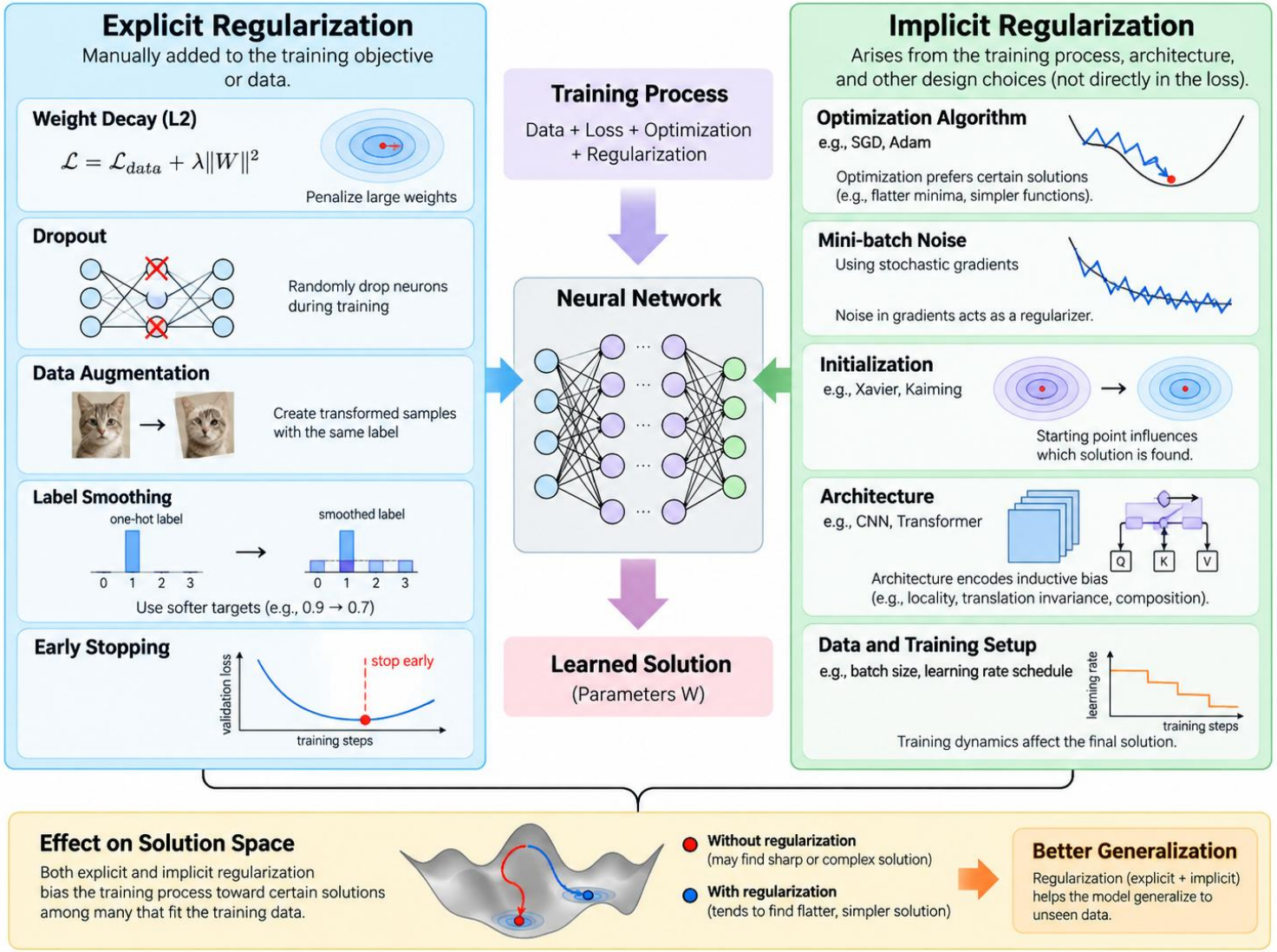


REGULARIZATION

- Regularization trong học sâu là một inductive bias.

- Explicit Regularization
  - Weight Decay
  - Dropout
  - Label Smoothing
  - Early Stopping

- Implicit Regularization
  - Architecture
  - Optimization
  - Initialization



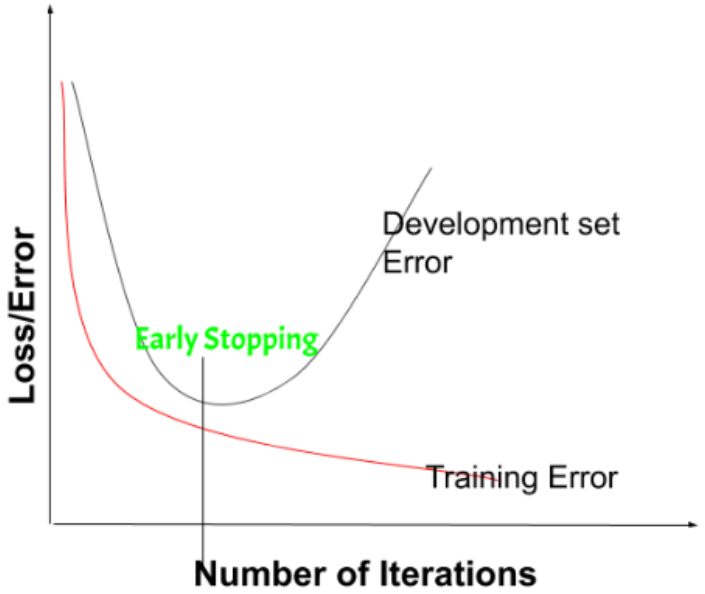
EARLY STOPPING

- Mô hình học sâu có xu hướng học các pattern có cấu trúc trước, rồi mới học nhiễu sau.

$$\text{Early Epochs} \rightarrow \text{Learn Signal}$$
$$\text{Late Epochs} \rightarrow \text{Fit Noise}$$

- Early Stopping hoạt động bằng cách ngừng học khi mà hiệu năng validation ngừng cải thiện.

- Có thể hiểu là bias với nghiệm học sớm.

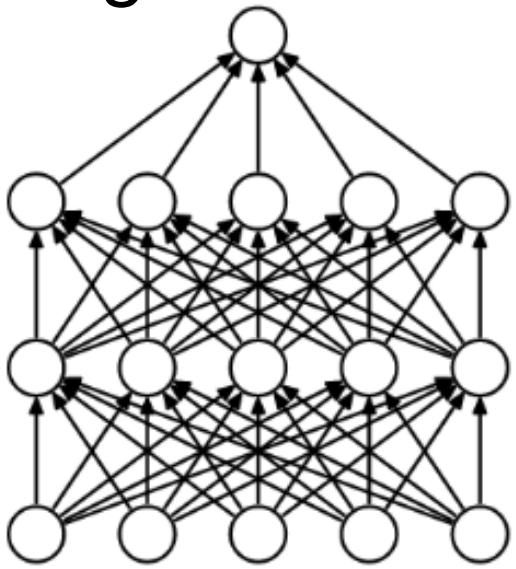


DROPOUT

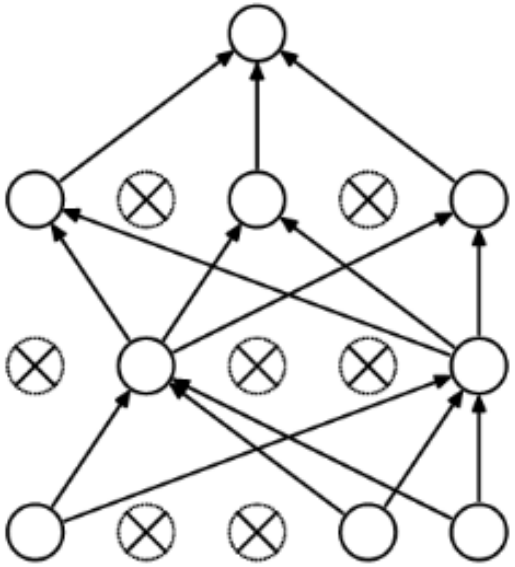
- Dropout là kỹ thuật tắt một phần nơ-ron trong quá trình huấn luyện
- Với một xác suất dropout p, ta thay mỗi nơ-ron h bằng

$$h' = \begin{cases} 0, & \text{với xác suất } p \\ \dfrac{h}{1-p}, & \text{với xác suất } 1-p \end{cases}$$

- Không dropout trong inference.



(a) Standard Neural Net



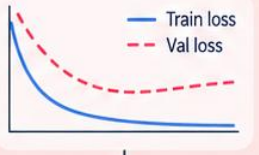
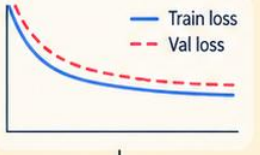
(b) After applying dropout.

CÁCH CHỌN REGULARIZATION

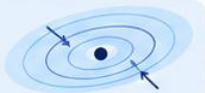
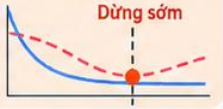
Cách chọn Regularization cho Deep Learning

1 Sơ đồ quyết định dựa trên train/validation

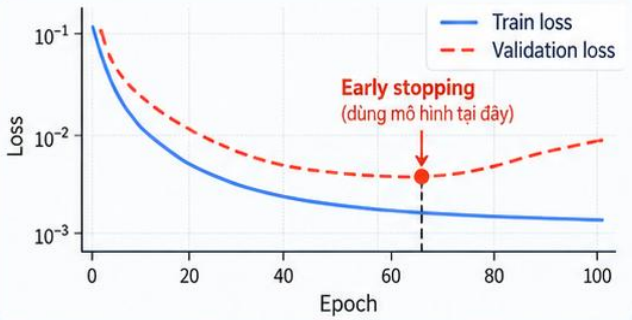
Quan sát train/validation
So sánh hiệu suất trên tập train và validation

Train tốt, Validation kém	Train kém, Validation cũng kém	Train tốt, Validation tốt
		
Train loss / Val loss	Train loss / Val loss	Train loss / Val loss
Overfitting	Underfitting	Giữ cấu hình hiện tại
Tăng regularization	Giảm regularization hoặc tăng capacity	

2 Chọn kỹ thuật regularization theo vấn đề

Weight Decay (L2 Regularization)	Khi weight quá lớn / cần bias về nghiệm norm nhỏ.	
Dropout	Khi mô hình phụ thuộc mạnh vào một số neuron.	
Data Augmentation	Khi dữ liệu ít, đặc biệt với ảnh.	
Early Stopping	Khi validation bắt đầu xấu đi dù train tiếp tục giảm.	 Dừng sớm
Label Smoothing	Khi cần làm mềm mục tiêu phân loại / có noise nhẹ ở nhãn.	 One-hot → Label smoothing

3 Minh họa training loss và validation loss



4 Gợi ý theo loại dữ liệu

Ảnh (CNN, Vision Transformer)	Augmentation + Weight Decay
Tabular / MLP (Dữ liệu dạng bảng)	Weight Decay + Dropout + Early Stopping
NLP / Transformer (Văn bản, ngôn ngữ)	Weight Decay + Dropout + Label Smoothing

5 Kết luận

1. Chọn regularization theo failure mode của mô hình.
2. Dùng validation set để tune độ mạnh regularization.
3. Không phải càng nhiều regularization càng tốt.

HỎI ĐÁP